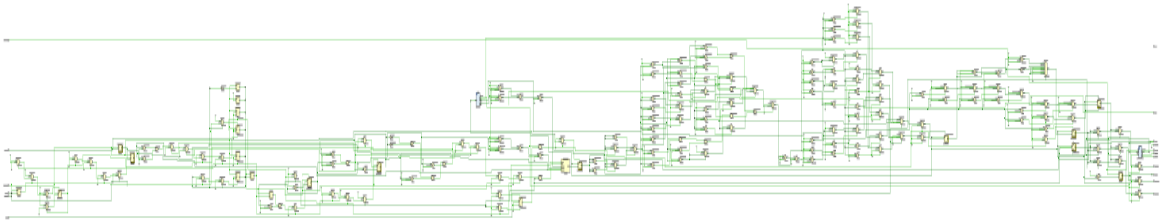
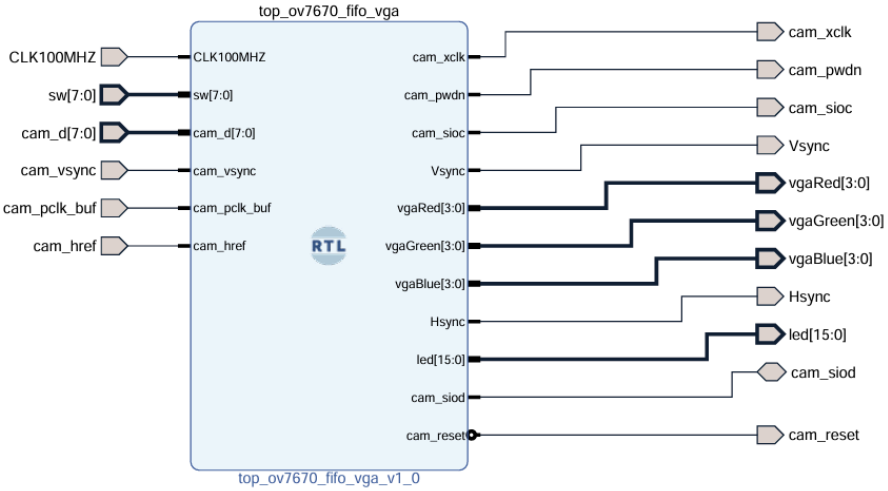
Group Member

Group name : AJ.JakeDotCom

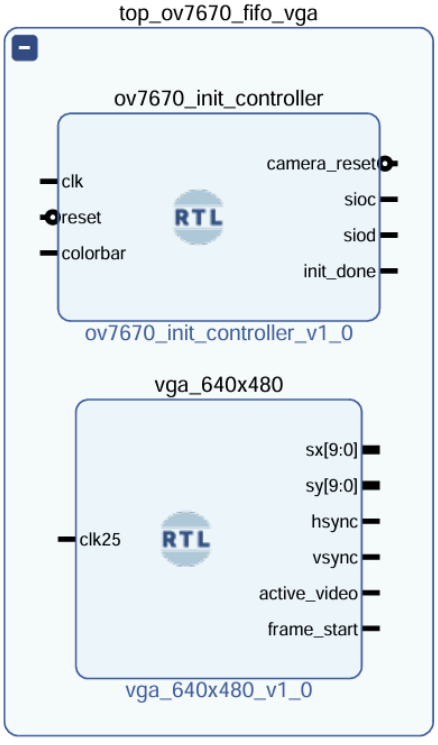
Name	Student Number
พอฤทัย ชินกาญจนโรจน์	6730356221
พารมี พีรภาพ	6730368821
ธีรภาส อภินันท์กูล	6731325021
วัทธิกร หนองน้ำขาว	6731349021

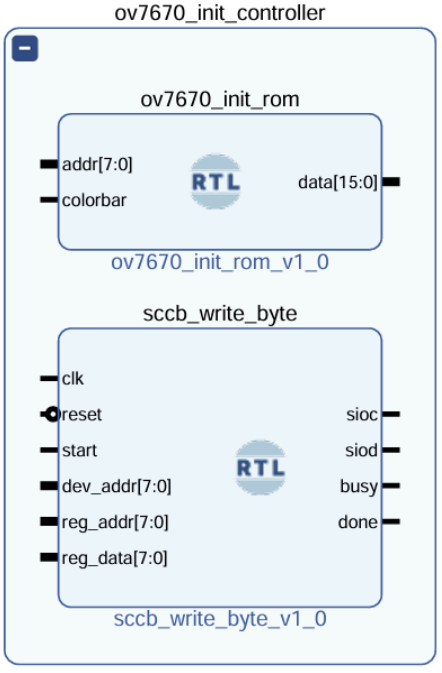
Overall Design Block Diagram



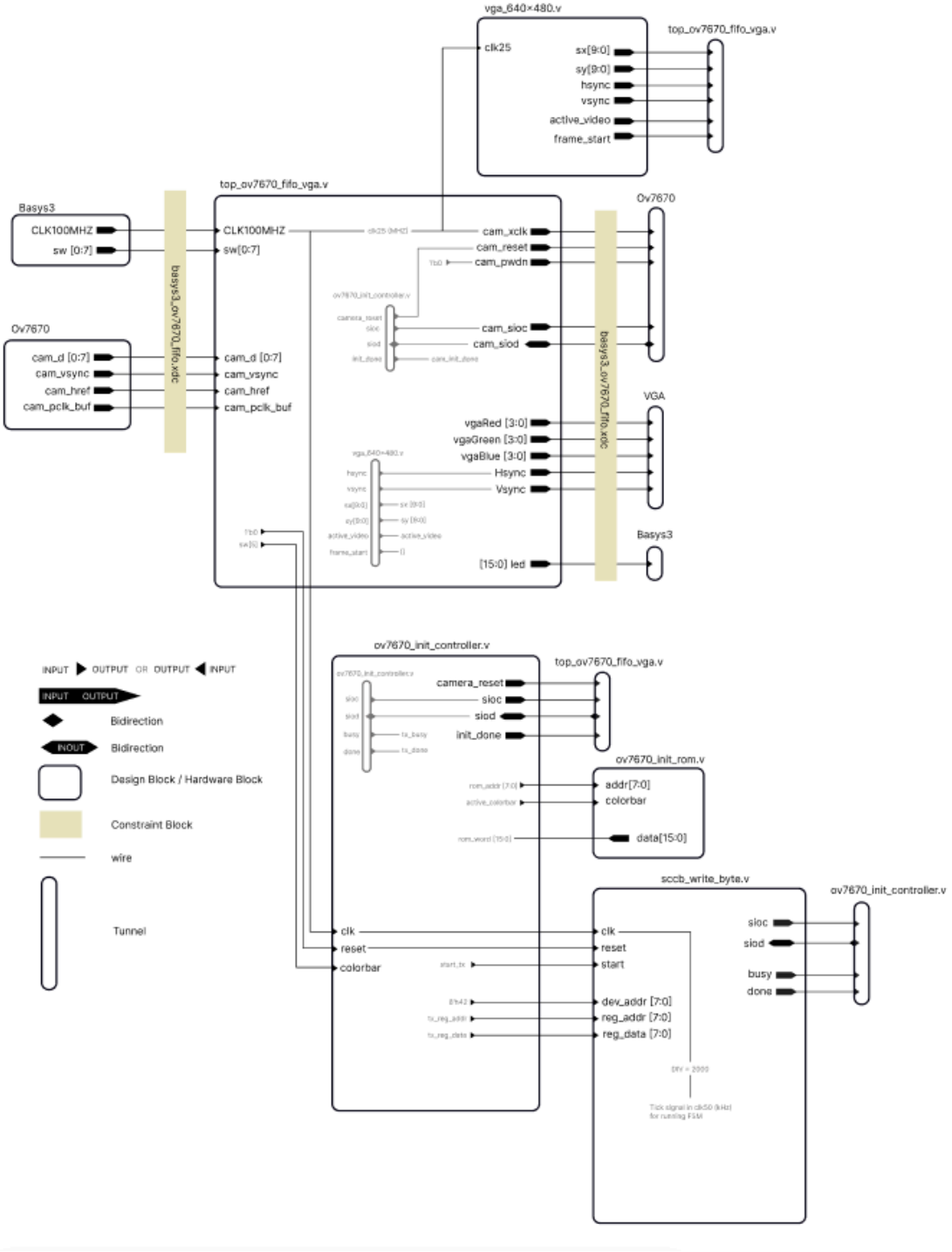


[Block&Schema]





Design Block Diagram (Detail)



[Figma]

Design Decision

1. Module Architecture

Our modules are

Module name	What it does
top_ov7670_fifo_vga.v	Main system integration
vga_640x480.v	VGA timing generator
sccb_write_byte.v	SCCB register writer
ov7670_init_controller.v	Camera initialization controller
ov7670_init_rom.v	Camera register configuration table

- Why did you structure your design this way?

- Answer: Because it separates each module to have a clear responsibility without using too many files. The VGA module only handles VGA timing. The SCCB module only handles one camera register write. The initialization controller only controls the startup sequence. The top module connects all parts together.

- What are the advantages of using this approach?

- Answer: This approach made it easier to debug each part separately, easier to simulate individual blocks and easier to understand the full system. These modules can be reused or modified independently and waveforms are easier to analyze because each module has a clear role.

2. Communication Protocols

We use SCCB for camera configuration

- Why did you choose this specific protocol for communication?

- Answer: Because the OV7670 camera expects SCCB for register access. SCCB is the protocol provided by the camera for configuring internal registers such as output format, color mode, reset, and test pattern settings. https://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf

- How does this protocol suit your design requirements?

- The OV7670 camera requires SCCB to set the camera into the correct operating mode. In this project, SCCB is used only during

initialization to write register values from the ROM into the camera.

3. **Clock and Data Transfer Rate**

- Why did you select this specific clock frequency?

- Answer: We selected a 25 MHz clock because it is suitable for 640x480 VGA timing. The Basys 3 board provides a 100 MHz clock, so we divide it by 4 to generate 25 MHz. This clock is close to the standard 25.175 MHz VGA pixel clock and is accepted by most VGA monitors. The same 25 MHz clock is also used as the OV7670 XCLK to simplify the design.

- How does the data rate impact system performance?

- Answer: The data rate affects how fast pixels can be captured and displayed. The VGA side must output one pixel every clock cycle to maintain stable video timing. On the camera side, the camera sends image data using its pixel clock, and the FPGA must capture the bytes fast enough to build pixels correctly. If the data rate is too slow, the display might become unstable, and if it is too fast, it might cause timing violations.

- Did you consider constraints such as FPGA timing, external device compatibility, or noise margins?

- Answer: Yes. The 25 MHz clock is low enough to be reliable on the Basys3 and VGA output pins, and still fast enough for 640x480 display. It is also suitable for the OV7670 camera input clock. Using this clock frequency helps reduce timing violations and improves signal reliability. The SCCB clock is further divided down from the 100 MHz system clock (by 2000) because camera register configuration does not require high speed and a slower SCCB clock gives better compatibility and noise margin.

4. **Resource Utilization**

- How does your design balance resource usage and performance?

- Answer: The design balances resource usage by storing a 320x240 image instead of a full 640x480 frame. This reduces BRAM usage while still allowing the image to fill the VGA screen by scaling each stored pixel into a 2x2 block. LUTs are used for control logic, state machines, RGB conversion, filter logic, and address calculation. Flip-flops are used for counters, synchronization, pixel capture registers, and state machine registers. BRAM is used for

the framebuffer because image storage requires much more memory than normal registers.

- Why did you optimize certain parts of the design?

- Answer: We optimized the design to reduce memory usage and LUT usage.
    i. The framebuffer stores RGB444 instead of RBG565 because the Basys3 VGA output only uses 4 bits per color channel.
    ii. The address calculation uses shifts instead of multiplication because 320 = 256 + 64, so the address can be calculated as (y << 8) + (y << 6) + x
    iii. The display scaling also uses bit shifting by reading sx[9:1] and sy[8:1], which is simpler than using a divider.

Implementation Detail

Module : top_ov7670_fifo_vga

The top_ov7670_fifo_vga module acts as the **top-level hardware architecture** for a real-time video capture and processing pipeline using a Basys 3 FPGA and an OV7670 camera.**Data Capture & Memory Storage:** It reads the incoming RGB565 pixel stream from the camera's clock domain, downsamples the video, converts it to 12-bit RGB444 format, and stores the image in an internal Block RAM (BRAM) framebuffer at a **320x240 resolution**.

- **Camera Configuration:** It initializes and configures the OV7670 camera module on startup using an instantiated SCCB (I2C-like) controller.
- **Data Capture & Memory Storage:** It reads the incoming RGB565 pixel stream from the camera's clock domain, downsamples the video, converts it to 12-bit RGB444 format, and stores the image in an internal Block RAM (BRAM) framebuffer at a **320x240 resolution**.
- **VGA Display Generation:** It generates standard **640x480 @ 60 Hz** VGA timing signals. To display the smaller 320x240 stored image, it performs real-time pixel-doubling (reading the same pixels and rows twice) to scale it up to the monitor's resolution.
- **Real-Time Image Processing:** It actively reads the physical slide switches (sw) on the Basys 3 board to apply real time video modifications. This includes basic color balancing, hardware test patterns, and selectable image filters (such as grayscale, color inversion, or individual Red/Green/Blue channel isolation).

```
`timescale 1ns / 1ps

// ============================================================================
// Top-level video pipeline สำหรับบอร์ด Basys 3 + กล้อง OV7670
// ทำหน้าที่เป็นศูนย์กลางเชื่อมต่อ Clock Domains และประมวลผลท่อข้อมูลภาพ (Pipeline)
// ============================================================================
module top_ov7670_fifo_vga (
    // ------------------------------------------------------------------
    // กลุ่มที่ 1: ขาพินเชื่อมต่อโลกภายนอก (I/O Ports)
    // ------------------------------------------------------------------
    // สัญญาณนาฬิกาหลักของบอร์ด 100 MHz และสวิตช์สไลด์ 8 ตัว
    input  wire CLK100MHZ,
    input  wire [7:0] sw,

    // สัญญาณที่รับเข้ามาจากกล้อง (Data 8 บิต, สัญญาณเริ่มเฟรม, ข้อมูลบรรทัด, นาฬิกากล้อง)
    input  wire [7:0] cam_d, // สายส่งข้อมูลสีขนาน 8 เส้น (โหมด RGB565 ส่ง 1 พิกเซล/16 บิต โดยแบ่งส่ง 2 รอบ รอบละ 8 บิต)
    input  wire       cam_vsync, // สัญญาณบอก "เริ่มเฟรมภาพใหม่" (กระเด้งเป็น 1 เพื่อให้ FPGA ดึงหัวเขียนกลับมุมซ้ายบนสุด)
    input  wire       cam_href, // สัญญาณบอก "ช่วงที่มีข้อมูลภาพ" (1 = ดึงสีไปเก็บได้, 0 = ช่วงขอบจอดำ/ปัดบรรทัด)
    input  wire       cam_pclk_buf, // นาฬิกาที่กล้องสร้าง เคาะบอก FPGA ว่าข้อมูลสีนิ่งแล้ว (ดึงเข้า Clock Buffer เพื่อให้นิ่งขึ้น)

    // สัญญาณควบคุมที่ส่งกลับไปให้กล้อง (นาฬิกาอ้างอิง, รีเซ็ต, ดึงลงกราวด์ให้ทำงานตลอด)
    output wire       cam_xclk, // นาฬิกาหลักความถี่ 25 MHz ที่ FPGA ต้องจ่ายไปเลี้ยงให้กล้องทำงาน
    output wire       cam_reset, // สายรีเซ็ตกล้อง (Active-Low: 0 = คืนค่าโรงงาน, 1 = ทำงานปกติ)
    output wire       cam_pwdn, // สายสั่งโหมดประหยัดไฟ (Active-High: ในโปรเจกต์นี้ดึงเป็น 0 ถาวรเพื่อให้กล้องตื่นตลอดเวลา)
```

```
    // สาย I2C/SCCB สำหรับเขียน Register ตั้งค่ากล้อง
    output wire       cam_sioc, // นาฬิกาจัดจังหวะส่งข้อมูลตั้งค่า (FPGA เป็นคนคุมจังหวะและส่งออกทางเดียว)
    inout  wire       cam_siod, // สายรับ-ส่งข้อมูล 2 ทิศทาง (FPGA ดึงสายเป็น 0 หรือปล่อยลอย High-Z เพื่อให้กล้องตอบ ACK)

    // สัญญาณส่งออกพอร์ต VGA (สีละ 4 บิต และสัญญาณกวาดหน้าจอ)
    output wire [3:0] vgaRed, // สายส่งข้อมูลสีแดง 4 บิต
    output wire [3:0] vgaGreen, // สายส่งข้อมูลสีเขียว 4 บิต
    output wire [3:0] vgaBlue, // สายส่งข้อมูลสีน้ำเงิน 4 บิต
    output wire       Hsync, // สัญญาณเคาะจบแนวนอน (บอกจอคอมฯ ให้ปัดหัวอ่านกลับไปซ้ายสุดเพื่อเริ่มบรรทัดใหม่)
    output wire       Vsync, // สัญญาณเคาะจบแนวตั้ง (บอกจอคอมฯ ให้ดึงหัวอ่านกลับไปบนสุดเพื่อเริ่มเฟรมภาพใหม่)

    // หลอดไฟ LED 16 ดวงสำหรับ Debug ดูสถานะฮาร์ดแวร์
    output wire [15:0] led
);

    // ============================================================================
    // กลุ่มที่ 2: สร้างสัญญาณนาฬิกา 25 MHz (Clock Divider)
    // ============================================================================
    // ใช้ตัวนับ 2 บิต (นับ 0-3) เพื่อหารความถี่จาก 100 MHz ลง 4 เท่า
    reg [1:0] div4 = 2'b00;
    always @(posedge CLK100MHZ) div4 <= div4 + 2'd1;

    // ดึงบิตสูงสุดมาใช้งาน จะได้ Clock 25 MHz สำหรับจอ VGA และให้กล้องอ้างอิง
    wire clk25 = div4[1];

    // จ่าย Clock 25 MHz ให้กล้อง และดึงขา Power Down (pwdn) เป็น 0 เพื่อปลุกกล้อง
    assign cam_xclk  = clk25;
    assign cam_pwdn  = 1'b0;

    // ============================================================================
    // กลุ่มที่ 3: เรียกใช้โมดูลตั้งค่ากล้อง (SCCB Init Controller)
    // ============================================================================
    wire cam_init_done; // ตัวแปรเก็บสถานะว่าตั้งค่ากล้องเสร็จหรือยัง

    // ส่งคำสั่งจาก ROM ไปตังค่ากล้อง OV7670 ให้เป็นโหมด RGB565 ผ่าน SCCB
    ov7670_init_controller init_u (
        .clk       (CLK100MHZ),
        .reset     (1'b0),
        .colorbar  (sw[5]),       // สับสวิตช์ 5 เพื่อบังคับกล้องส่งแถบสีทดสอบ
        .camera_reset (cam_reset),
        .sioc      (cam_sioc),
        .siod      (cam_siod),
        .init_done (cam_init_done) // แจ้งกลับเมื่อตั้งค่าเสร็จ วงจรรับภาพถึงจะเริ่มทำงาน
    );
```

```
// ================================================================================
// กลุ่มที่ 4: เรียกใช้โมดูลสร้างพิกัดหน้าจอ VGA
// ================================================================================
wire [9:0] sx, sy;       // ตัวเก็บพิกัดแกน X และ Y ปัจจุบันของหัวอ่านจอภาพ
wire       active_video; // สัญญาณบอกว่าหัวอ่านอยู่ในพื้นที่แสดงภาพ (ไม่ใช่ขอบดำ)

// สร้างจังหวะกวาดจอ (Hsync/Vsync) สำหรับความละเอียด 640x480
vga_640x480 vga_u (
    .clk25        (clk25),
    .sx           (sx),
    .sy           (sy),
    .hsync        (Hsync),
    .vsync        (Vsync),
    .active_video (active_video),
    .frame_start  ()
);

// ================================================================================
// กลุ่มที่ 5: หน่วยความจำภาพ (Block RAM Framebuffer)
// ================================================================================
localparam FRAME_WIDTH  = 320;
localparam FRAME_HEIGHT = 240;
localparam FRAME_PIXELS = 76800; // จำนวนพิกเซลทั้งหมด (320 * 240)

// สร้างอาร์เรย์เก็บข้อมูลสี 12 บิต จำนวน 76,800 ช่อง
// คำสั่ง ram_style="block" บังคับให้ Vivado สังเคราะห์เป็น BRAM ภายในชิป
(* ram_style = "block" *) reg [11:0] framebuffer [0:FRAME_PIXELS-1];

// ================================================================================
// กลุ่มที่ 6: ตัวแปรสำหรับวงจรดักจับภาพ (Capture Domain Registers)
// ================================================================================
// ตัวแปรกลุ่มนี้ใช้ตรวจสอบว่ามีสัญญาณนาฬิกาและข้อมูลส่งมาจากกล้องจริงๆ หรือไม่
reg pclk_activity = 1'b0;
reg pclk_seen     = 1'b0;
reg href_seen     = 1'b0;
reg vsync_seen    = 1'b0;
reg frame_seen    = 1'b0;
reg pixel_seen    = 1'b0;
reg frame_ready   = 1'b0;

// ตัวแปรสำหรับประกอบร่างพิกเซล
reg       byte_phase = 1'b0; // ใช้บอกว่ากำลังรับข้อมูล 8 บิตชุดแรกหรือชุดหลัง (0 หรือ 1)
reg       href_d = 1'b0;     // เก็บค่า href รอบที่แล้วเพื่อจับจังหวะขอบขาลง (จบบรรทัด)
reg [7:0] byte_hi = 8'h00;   // เก็บข้อมูล 8 บิตชุดแรกทดไว้รอชุดที่สอง
reg [9:0] cap_x = 10'd0;     // ตัวนับพิกัดแนวนอนของกล้อง (0-639)
reg [8:0] cap_y = 9'd0;      // ตัวนับพิกัดแนวตั้งของกล้อง (0-479)
```

```
// ================================================================================
// ด่านที่ 6.1: ล็อกสัญญาณกันรบกวน (Latching on Negedge PCLK)
// ================================================================================
reg       latched_vsync = 1'b0;
reg       latched_href = 1'b0;
reg [7:0] latched_d = 8'h00;

// ทำงานตอนขอบขาลงของนาฬิกากล้อง (ช่วงที่สัญญาณสายไฟนิ่งที่สุด)
// ดึงค่ามาพักไว้ในตลับ Register เพื่อกันภาพล้มจากสัญญาณกวน (Noise)
always @(negedge cam_pclk_buf) begin
    latched_vsync <= cam_vsync;
    latched_href <= cam_href;
    latched_d <= cam_d;
end

// ================================================================================
// ด่านที่ 6.2: คำนวณที่อยู่เขียนหน่วยความจำ (Write Address)
// ================================================================================
// ตัดบิตล่างสุดทิ้ง ([9:1]) มีค่าเท่ากับหาร 2
// เป็นการลดขนาดภาพ (Downsampling) จาก 640x480 เหลือ 320x240
wire [8:0] cap_dst_x = cap_x[9:1];
wire [7:0] cap_dst_y = cap_y[8:1];

// คำนวณตำแหน่ง 1 มิติด้วยสูตร Address = (Y * 320) + X
// ใช้เทคนิค Bit Shift แทนวงจรคูณ: (Y * 256) + (Y * 64) + X ประหยัด Logic Gate
wire [16:0] wr_addr =
    ({9'd0, cap_dst_y} << 8) + ({9'd0, cap_dst_y} << 6) + {8'd0, cap_dst_x};

// ================================================================================
// ด่านที่ 6.3: วงจรเก็บเกี่ยวข้อมูลภาพ (Core Capture Logic on Posedge PCLK)
// ================================================================================
// ทำงานตอนขอบขาขึ้นของนาฬิกากล้อง
always @(posedge cam_pclk_buf) begin
    pclk_activity <= ~pclk_activity; // กระพริบเพื่อเช็คว่า Clock ทำงาน
    pclk_seen <= 1'b1;
    href_d <= latched_href;          // อัปเดต href_d ของรอบที่แล้ว

    if (latched_href)  href_seen <= 1'b1;
    if (latched_vsync) vsync_seen <= 1'b1;

    // สถานะที่ 1: ถ้ากล้องยัง Init ไม่เสร็จ ให้รีเซ็ตระบบรับภาพรอไปก่อน
    if (!cam_init_done) begin
        byte_phase <= 1'b0;
        cap_x <= 10'd0;
        cap_y <= 9'd0;
        frame_ready <= 1'b0;
```

```
        pixel_seen <= 1'b0;

    // สถานะที่ 2: เจอ Vsync (เริ่มภาพใหม่) ให้ดึงหัวเขียนกลับไปมุมซ้ายบน
    end else if (latched_vsync) begin
        byte_phase <= 1'b0;
        cap_x <= 10'd0;
        cap_y <= 9'd0;
        // ถ้ารอบที่แล้วมีพิกเซลเข้ามาจริง แจ้งหน้าจอ VGA ว่ามีภาพพร้อมให้ดึงไปวาดแล้ว
        if (pixel_seen) begin
            frame_seen <= 1'b1;
            frame_ready <= 1'b1;
        end
        pixel_seen <= 1'b0;

    // สถานะที่ 3: กำลังรับข้อมูลสี (Href เป็น Logic 1)
    end else if (latched_href) begin

        // จังหวะรับไบต์แรก (ครึ่งบน)
        if (!byte_phase) begin
            byte_hi <= latched_d; // เก็บข้อมูลทดไว้ก่อน
            byte_phase <= 1'b1;   // สลับเฟสเพื่อรอรับครึ่งหลัง

        // จังหวะรับไบต์ที่สอง (ครึ่งล่าง) นำมาประกอบกัน
        end else begin
            byte_phase <= 1'b0; // สลับเฟสกลับ

            // ตรวจสอบว่าพิกัดไม่ออกนอกจอ
            if ((cap_x < 10'd640) && (cap_y < 9'd480)) begin

                // เงื่อนไข Downsampling: จะเขียนลง BRAM ก็ต่อเมื่อพิกัด X และ Y เป็น "เลขคู่"
                // (!cap_x[0] หมายถึงบิตล่างสุดเป็น 0) ข้ามพิกเซลช่องคี่ทิ้งไปเลย
                if (!cap_x[0] && !cap_y[0] && (cap_dst_x < FRAME_WIDTH) && (cap_dst_y < FRAME_HEIGHT)) begin
                    // นำ 2 ไบต์มาแปลงสีเหลือ 12 บิต แล้วยัดลงล็อกเกอร์ BRAM
                    framebuffer[wr_addr] <= rgb565_to_rgb444(byte_hi, latched_d);
                    if ((byte_hi != 8'h00) || (latched_d != 8'h00))
                        pixel_seen <= 1'b1; // ตรวจสอบว่าภาพไม่ใช่สีดำล้วน
                end
                cap_x <= cap_x + 10'd1; // ขยับหัวเขียนแนวนอนไปทางขวา 1 ช่อง
            end
        end

    // สถานะที่ 4: จบบรรทัด (Href ตกจาก 1 เป็น 0)
    end else begin
        byte_phase <= 1'b0;
        cap_x <= 10'd0; // ดึงหัวเขียนกลับไปซ้ายสุดของจอ

        // ใช้เทคนิคตรวจขอบขาลง (Falling edge detection) ของ Href
        // ถ้ารอบที่แล้วเป็น 1 รอบนี้เป็น 0 ให้ปัดหัวเขียนลงมาบรรทัดล่าง
```

```
        if (href_d && !latched_href && (cap_y < 9'd479))
            cap_y <= cap_y + 9'd1;
    end
end

// ============================================================================
// กลุ่มที่ 7: คำนวณที่อยู่เพื่ออ่านข้อมูลไปแสดงจอ VGA (Read Address)
// ============================================================================
// นำพิกัดจอภาพ 640x480 มาหาร 2 (ตัดบิตล่างทิ้ง)
// ทำให้เกิดเอฟเฟกต์ Pixel Doubling ขยายภาพ 320x240 ให้เต็มจออัตโนมัติ
wire [8:0] src_x = sx[9:1];
wire [7:0] src_y = sy[8:1];

// คำนวณตำแหน่ง 1 มิติด้วยสูตร (Y * 320) + X สำหรับดึงข้อมูลออกจาก BRAM
wire [16:0] rd_addr =
    ({9'd0, src_y} << 8) + ({9'd0, src_y} << 6) + {8'd0, src_x};

// เตรียมตัวแปร Pipeline เป็นจุดพักข้อมูลก่อนส่งออกจอ
reg [11:0] camera_rgb = 12'h000;   // รับภาพดิบจาก BRAM
reg [11:0] balanced_rgb = 12'h000; // รับภาพที่แก้สีเพี้ยนแล้ว
reg [11:0] filtered_rgb = 12'h000; // รับภาพที่ใส่ฟิลเตอร์ผู้ใช้แล้ว
reg [11:0] test_rgb = 12'h000;     // เก็บภาพแถบสีจำลอง
reg [11:0] live_rgb = 12'h000;     // เก็บภาพสดไม่ผ่าน BRAM (Debug)

// ============================================================================
// กลุ่มที่ 8: ฟังก์ชันย่อยคณิตศาสตร์และประมวลผลภาพ (Helper Functions)
// ============================================================================

// 8.1 ฟังก์ชันลดบิตสีจาก 16 บิต (RGB565) ให้เป็น 12 บิต (RGB444) เพื่อประหยัด BRAM
function [11:0] rgb565_to_rgb444;
    input [7:0] hi;
    input [7:0] lo;
    reg [3:0] r4; reg [3:0] g4; reg [3:0] b4;
    begin
        r4 = hi[7:4];           // ดึงแดง 4 บิตบน
        g4 = {hi[2:0], lo[7]};  // ประกอบเขียว 4 บิตบน
        b4 = lo[4:1];           // ดึงน้ำเงิน 4 บิตบน
        rgb565_to_rgb444 = {r4, g4, b4};
    end
endfunction

// 8.2 ฟังก์ชันใส่ฟิลเตอร์วิดีโอ (เลือกได้จากสวิตช์ sw[4:2])
function [11:0] apply_filter;
    input [11:0] rgb;
    input [2:0] mode;
    reg [3:0] gray;
    begin
        // สร้างสีเทาโดยนำ R+G+B แล้วหาร 4 (Shift right 2)
```

```
        gray = ({2'b00, rgb[11:8]} + {2'b00, rgb[7:4], 1'b0} + {2'b00, rgb[3:0]}) >> 2;
        case (mode)
            3'b001: apply_filter = {gray, gray, gray};      // โหมดขาวดำ
            3'b010: apply_filter = ~rgb;                    // โหมดสีตรงข้าม (Invert)
            3'b011: apply_filter = {rgb[11:8], 8'h00};      // ดึงมาแค่ช่องสีแดง
            3'b100: apply_filter = {4'h0, rgb[7:4], 4'h0};  // ดึงมาแค่ช่องสีเขียว
            3'b101: apply_filter = {8'h00, rgb[3:0]};       // ดึงมาแค่ช่องสีน้ำเงิน
            default: apply_filter = rgb;                    // ภาพปกติ
        endcase
    end
endfunction

// 8.3 ฟังก์ชันลบเลขแบบอิ่มตัว (Saturating Subtract) ป้องกันค่าติดลบแล้ววนกลับไปค่าสูงสุด
function [3:0] sub_sat4;
    input [3:0] value;
    input [3:0] amount;
    begin
        sub_sat4 = (value > amount) ? (value - amount) : 4'h0;
    end
endfunction

// 8.4 ฟังก์ชันบวกเลขแบบอิ่มตัว (Saturating Add) ป้องกันค่าเกิน 15 (0xF)
function [3:0] add_sat4;
    input [3:0] value;
    input [3:0] amount;
    reg [4:0] sum;
    begin
        sum = {1'b0, value} + {1'b0, amount};
        add_sat4 = sum[4] ? 4'hF : sum[3:0];
    end
endfunction

// 8.5 ฟังก์ชันปรับแก้สีเพี้ยน (แก้ปัญหาจุดแดงในที่มืดของเซนเซอร์ราคาถูก)
function [11:0] color_balance;
    input [11:0] rgb;
    reg [3:0] r; reg [3:0] g; reg [3:0] b;
    reg [3:0] max_ch; reg [5:0] sum_ch;
    reg [4:0] r_ext; reg [4:0] g_ext; reg [4:0] b_ext;
    reg       red_dominant; reg       low_light; reg       purple_dominant;
    begin
        r = rgb[11:8]; g = rgb[7:4]; b = rgb[3:0];
        max_ch = (r > g) ? ((r > b) ? r : b) : ((g > b) ? g : b);
        sum_ch = {2'b00, r} + {2'b00, g} + {2'b00, b};
        r_ext = {1'b0, r}; g_ext = {1'b0, g}; b_ext = {1'b0, b};

        // เช็คว่าพิกเซลนี้มีสีไหนโดดเด่น หรือเป็นที่มืด
        red_dominant = (r_ext > g_ext + 5'd1) && (r_ext > b_ext + 5'd1);
        low_light = (max_ch < 4'hB);
```

```
        purple_dominant = (r_ext > g_ext + 5'd1) && (b_ext > g_ext + 5'd1);

        // ถ้ามืดมากและมีจุดแดงโผล่มา ให้กดสีให้ดำสนิทไปเลย
        if ((max_ch < 4'h2) || (sum_ch < 6'd5) || (low_light && (g < 4'h3) && (b < 4'h3) && red_dominant)) begin
            color_balance = 12'h000;
        // ถ้าภาพติดม่วง ให้ลดแดงกับน้ำเงิน เพิ่มเขียว
        end else if (purple_dominant) begin
            r = sub_sat4(r, 4'h1); b = sub_sat4(b, 4'h1);
            if (g < 4'hE) g = add_sat4(g, 4'h1);
            color_balance = {r, g, b};
        // ถ้าภาพติดแดง ให้ลดแดง
        end else if (red_dominant) begin
            if (max_ch < 4'hC) r = sub_sat4(r, 4'h2);
            else r = sub_sat4(r, 4'h1);
            color_balance = {r, g, b};
        end else begin
            color_balance = {r, g, b};
        end
    end
endfunction

// 8.6 ฟังก์ชันลดความอิ่มตัวของสี (ให้ภาพดูเป็นธรรมชาติขึ้น)
function [11:0] soft_desaturate;
    input [11:0] rgb;
    reg [3:0] r; reg [3:0] g; reg [3:0] b; reg [3:0] gray;
    reg [4:0] r_ext; reg [4:0] g_ext; reg [4:0] b_ext; reg [4:0] gray_ext;
    begin
        r = rgb[11:8]; g = rgb[7:4]; b = rgb[3:0];
        gray = ({2'b00, r} + {2'b00, g, 1'b0} + {2'b00, b}) >> 2;
        r_ext = {1'b0, r}; g_ext = {1'b0, g}; b_ext = {1'b0, b}; gray_ext = {1'b0, gray};

        // ดึงสี R, G, B แต่ละช่องให้เข้าใกล้ค่าสีเทา (Gray) ของพิกเซลนั้นๆ เล็กน้อย
        if (r_ext > gray_ext + 5'd1) r = sub_sat4(r, 4'h1);
        else if (gray_ext > r_ext + 5'd1) r = add_sat4(r, 4'h1);

        if (g_ext > gray_ext + 5'd1) g = sub_sat4(g, 4'h1);
        else if (gray_ext > g_ext + 5'd1) g = add_sat4(g, 4'h1);

        if (b_ext > gray_ext + 5'd1) b = sub_sat4(b, 4'h1);
        else if (gray_ext > b_ext + 5'd1) b = add_sat4(b, 4'h1);

        soft_desaturate = {r, g, b};
    end
endfunction

// ============================================================================
// กลุ่มที่ 9: ท่อประมวลผลก่อนส่งออกจอ (VGA Output Pipeline)
// ============================================================================
```

```
// ทำงานภายใต้ความเร็วจอภาพ (25 MHz)
always @(posedge clk25) begin

    // 9.1 จำลองหน้าจอแถบสี (อิงพิกัดจากแกน X) เอาไว้เทสสาย VGA
    if (sx < 80)       test_rgb <= 12'hF00;
    else if (sx < 160) test_rgb <= 12'h0F0;
    else if (sx < 240) test_rgb <= 12'h00F;
    else if (sx < 320) test_rgb <= 12'hFF0;
    else if (sx < 400) test_rgb <= 12'h0FF;
    else if (sx < 480) test_rgb <= 12'hF0F;
    else if (sx < 560) test_rgb <= 12'hFFF;
    else               test_rgb <= 12'h888;

    // 9.2 ดึงสัญญาณสดจากสายไฟกล้องมาประกอบเป็นสี (Live Feed) ข้าม BRAM ไปเลย
    live_rgb <= {
        cam_vsync ? 4'hF : cam_d[7:4],
        cam_href  ? 4'hF : cam_d[3:0],
        pclk_seen ? 4'hF : 4'h0
    };

    // 9.3 ท่อประมวลผลหลัก (ทำงานเฉพาะตอนหัวอ่านอยู่ในหน้าจอแสดงผล)
    if (active_video && frame_ready) begin
        camera_rgb <= framebuffer[rd_addr]; // ขั้น 1: ดึงภาพจาก BRAM

        // ขั้น 2: ถ้าไม่ได้เทสแถบสีกล้อง (sw[5]) ให้รันผ่านฟังก์ชันแก้สี
        balanced_rgb <= sw[5] ? camera_rgb : soft_desaturate(color_balance(camera_rgb));

        // ขั้น 3: ใส่ฟิลเตอร์สีตามสวิตช์ผู้ใช้ (sw[4:2])
        filtered_rgb <= apply_filter(balanced_rgb, sw[4:2]);
    end else begin
        // ถ้าหัวอ่านอยู่นอกจอภาพ ให้ดับสีเป็นสีดำ (ป้องกันจอขึ้นเส้นประหลาด)
        camera_rgb <= 12'h000;
        balanced_rgb <= 12'h000;
        filtered_rgb <= 12'h000;
    end
end

// ============================================================================
// กลุ่มที่ 10: วงจรเลือกสัญญาณออกจอภาพ (Output Multiplexers)
// ============================================================================
// ใช้คำสั่งเงื่อนไข ? : ซ้อนกันเพื่อทำหน้าที่เหมือนวงจร MUX แบบฮาร์ดแวร์
// ลำดับความสำคัญ:
// 1. ถ้าอยู่นอก active_video ส่งสี 0 (ดำ) ทันที
// 2. ถ้า sw[0]=1 ส่งภาพแถบสีจำลอง test_rgb (ใช้ได้แม้ไม่ต่อกล้อง)
// 3. ถ้า sw[1]=1 ส่งภาพดิบ live_rgb
// 4. นอกเหนือจากนั้น ส่งภาพที่ผ่านฟิลเตอร์สมบูรณ์ filtered_rgb
assign vgaRed   = active_video ? (sw[0] ? test_rgb[11:8] : (sw[1] ? live_rgb[11:8] : filtered_rgb[11:8])) : 4'h0;
assign vgaGreen = active_video ? (sw[0] ? test_rgb[7:4]  : (sw[1] ? live_rgb[7:4]  : filtered_rgb[7:4]))  : 4'h0;
```

```
    assign vgaBlue  = active_video ? (sw[0] ? test_rgb[3:0]  : (sw[1] ? live_rgb[3:0]  : filtered_rgb[3:0]))  : 4'h0;

    // ================================================================================
    // กลุ่มที่ 11: แผนที่ดวงไฟ LED (Hardware Debug Map)
    // ================================================================================
    // เชื่อมต่อสถานะต่างๆ ลงไฟ LED บนบอร์ดเพื่อให้วิศวกรมองเห็นปัญหาได้ทันที
    assign led[0]  = cam_init_done; // สว่าง = กล้องคอนฟิกเสร็จแล้ว
    assign led[1]  = vsync_seen;    // สว่าง = รับรู้สัญญาณเฟรมใหม่
    assign led[2]  = href_seen;     // สว่าง = รับรู้สัญญาณข้อมูลบรรทัด
    assign led[3]  = pclk_activity; // กระพริบรัว = สัญญาณนาฬิกากล้องเดิน
    assign led[4]  = cam_vsync;     // ไฟดิดดับตามจังหวะขึ้นเฟรมใหม่
    assign led[5]  = cam_href;      // ไฟดิดดับตามจังหวะวาดบรรทัด
    assign led[6]  = cam_d[7];      // ดึงบิตสีบิตนึงมาเทสการขยับ
    assign led[7]  = sw[0];         // ดูสถานะสวิตช์โหมด
    assign led[8]  = frame_seen;    // สว่าง = เก็บภาพลง BRAM สำเร็จแล้ว
    assign led[9]  = pixel_seen;    // สว่าง = กล้องไม่ได้ส่งภาพมาแค่มืดๆ
    assign led[10] = cam_href;
    assign led[11] = pclk_seen;
    assign led[12] = sw[1];
    assign led[14:13] = sw[7:6];
    assign led[15] = sw[5];

endmodule
```

Module : ov7670_init_controller

The ov7670_init_controller module acts as a **hardware configuration sequencer** for the OV7670 camera.

- **Camera Reset & Stabilization:** It asserts the hardware reset line (camera_reset) to initialize the camera and waits for a specific number of clock cycles (SETTLE_CYCLES) to let the camera stabilize before sending commands.
- **ROM Reading:** It reads predefined OV7670 configuration data (register addresses and values) sequentially from an instantiated ROM module (ov7670_init_rom).
- **SCCB Transmission:** It takes the data read from the ROM and transmits it to the camera using the Serial Camera Control Bus (SCCB) protocol via the sccb_write_byte module.
- **Delay & Completion Handling:** It interprets special ROM markers, such as 0xFFFE to pause and wait (allowing the camera to process a command) and 0xFFFF to stop the sequence and assert the init_done signal.
- **Colorbar Reconfiguration:** It monitors the colorbar input. If toggled, it automatically resets the state machine and re-runs the entire initialization sequence to switch the camera into or out of its internal test pattern mode.

```
// ================================================================================
// โมดูลตัวควบคุมลำดับการตั้งค่ากล้อง (OV7670 Configuration Sequencer)
//
// หน้าที่หลัก:
// 1. ทำการ Hardware Reset ตัวกล้อง
// 2. หน่วงเวลาให้วงจรภายในกล้องพร้อมทำงาน
// 3. อ่านค่าจาก ROM แล้วส่งผ่านวงจร SCCB ไปยังกล้องทีละบรรทัดจนจบ
// 4. หากมีการสับสวิตช์ Colorbar จะทำการรีเซ็ตวงจรเพื่อตั้งค่าใหม่ทั้งหมด (On-the-fly)
// ================================================================================
module ov7670_init_controller #(
    // พารามิเตอร์สำหรับหน่วงเวลา (Delay)
    // 200,000 cycles ที่ Clock 100MHz = หน่วงเวลา 2 มิลลิวินาที (ms)
    // เพื่อให้เวลาวงจรอิเล็กทรอนิกส์ในกล้องเคลียร์ประจุและตั้งสถิก่อนรับคำสั่ง
    parameter integer SETTLE_CYCLES = 200000,
    parameter integer SCCB_DIV = 2000 // ตัวหารความถี่สำหรับ SCCB
) (
    input  wire clk,         // สัญญาณนาฬิกาหลัก (100 MHz)
    input  wire reset,       // สัญญาณรีเซ็ตโมดูลนี้ (ในโปรเจกต์นี้ถูกผูกกับ 0 ไว้)
    input  wire colorbar,    // สัญญาณจากสวิตช์บนบอร์ด เพื่อเปิด/ปิดแถบสีทดสอบ
    output reg  camera_reset = 1'b0, // ขาที่ต่อตรงไปที่พิน RESET ของตัวกล้อง (Active-Low)
    output wire sioc,        // สัญญาณ Clock ของ SCCB
    inout  wire siod,        // สัญญาณ Data ของ SCCB
    output reg  init_done = 1'b0     // ธง (Flag) ประกาศว่าตั้งค่ากล้องเสร็จสมบูรณ์แล้ว
);

    // ================================================================================
    // ส่วนที่ 1: ตัวแปรภายในสำหรับเชื่อมต่อและนับเวลา
    // ================================================================================
    reg [19:0] wait_ctr = 20'd0; // ตัวนับเวลาขนาด 20 บิต (นับได้สูงสุดประมาณ 1 ล้าน)
```

```
// กลุ่มตัวแปรสำหรับสื่อสารกับลูกน้องคนที่ 1 (SCCB Transmitter)
reg start_tx = 1'b0;         // สั่งให้ SCCB เริ่มส่งข้อมูล
reg [7:0] tx_reg_addr = 8'h00; // ตำแหน่งรีจิสเตอร์ที่จะฝากส่ง
reg [7:0] tx_reg_data = 8'h00; // ข้อมูลที่จะฝากส่ง
wire tx_busy;                // สถานะจาก SCCB บอกว่า "สายไม่ว่าง"
wire tx_done;                // สถานะจาก SCCB บอกว่า "ส่งเสร็จแล้ว"

// กลุ่มตัวแปรสำหรับสื่อสารกับลูกน้องคนที่ 2 (ROM)
reg [7:0] rom_addr = 8'd0;   // บรรทัดของ ROM ที่ต้องการอ่าน
wire [15:0] rom_word;        // ข้อมูล 16 บิตที่ ROM คายออกมาให้

// ตัวแปรสำหรับวงจรซิงโครไนซ์สวิตช์ (Switch Synchronizer)
reg colorbar_d = 1'b0;       // เก็บค่าสถานะสวิตช์ในอดีต (เพื่อหาขอบสัญญาณ)
reg active_colorbar = 1'b0;  // สถานะ Colorbar ที่จะส่งไปให้ ROM ตัดสินใจ
reg colorbar_meta = 1'b0;    // ตัวกันกระแทกชั้นที่ 1 (Metastability filter)
reg colorbar_sync = 1'b0;    // ตัวกันกระแทกชั้นที่ 2 (ทำให้สัญญาณนิ่ง 100%)

// ================================================================================
// ส่วนที่ 2: การเรียกใช้ลูกน้อง (Sub-module Instantiation)
// ================================================================================

// ลูกน้องคนที่ 1: แผนกคลังข้อมูล (ROM)
ov7670_init_rom rom_u (
    .addr     (rom_addr),        // ส่งเลขบรรทัดไปถาม
    .colorbar (active_colorbar), // ส่งสถานะสวิตช์ไปให้เลือกค่า
    .data     (rom_word)         // รับข้อมูล 16 บิตกลับมา
);

// ลูกน้องคนที่ 2: แผนกส่งสาร (SCCB Engine)
sccb_write_byte #(
    .DIV(SCCB_DIV)
) sccb_u (
    .clk        (clk),
    .reset      (reset),
    .start      (start_tx),     // สัญญาณสั่นไก
    .dev_addr   (8'h42),        // ID ของกล้อง OV7670 สำหรับเขียนข้อมูล
    .reg_addr   (tx_reg_addr),  // ที่อยู่รีจิสเตอร์
    .reg_data   (tx_reg_data),  // ข้อมูลที่จะเขียน
    .sioc       (sioc),
    .siod       (siod),
    .busy       (tx_busy),      // รอรับสถานะไม่ว่าง
    .done       (tx_done)       // รอรับสถานะส่งเสร็จ
);

// ================================================================================
// ส่วนที่ 3: เครื่องจักรสถานะ (Finite State Machine - FSM)
// ================================================================================
```

```
// ประกาศชื่อสถานะต่างๆ ให้โค้ดอ่านง่าย (เข้ารหัสแบบตัวเลข 3 บิต)
localparam S_RESET = 3'd0; // สถานะ: กดปุ่มรีเซ็ตกล้องแช่ไว้
localparam S_WAIT  = 3'd1; // สถานะ: ปล่อยปุ่มรีเซ็ตและรอให้กล้องฟื้นตัว
localparam S_LOAD  = 3'd2; // สถานะ: ดึงค่าสั่งจาก ROM
localparam S_START = 3'd3; // สถานะ: สั่ง SCCB ยิงข้อมูล
localparam S_NEXT  = 3'd4; // สถานะ: รอ SCCB ทำงานเสร็จ
localparam S_DONE  = 3'd5; // สถานะ: จบภารกิจ

reg [2:0] state = S_RESET;

always @(posedge clk) begin
    // ---------------------------------------------------------------------
    // วงจรป้องกันสัญญาณสั่นจากสวิตช์ (2-Stage Synchronizer)
    // สวิตช์มือคนเราเวลากด มันจะมีจังหวะหน้าสัมผัสเด้ง (Bouncing) และเกิดสภาวะลอยตัว
    // การเอาผ่าน Flip-Flop 2 ตัวแบบนี้ จะช่วยให้ฮาร์ดแวร์ไม่รวนครับ
    // ---------------------------------------------------------------------
    colorbar_meta <= colorbar;
    colorbar_sync <= colorbar_meta;

    // ถ้ามีการสั่ง Reset ระดับ System
    if (reset) begin
        state <= S_RESET;
        wait_ctr <= 0;
        rom_addr <= 0;
        start_tx <= 0;
        init_done <= 0;
        camera_reset <= 1'b0;
        active_colorbar <= colorbar_sync;
        colorbar_d <= colorbar_sync;
    end else begin
        start_tx <= 0; // เคลียร์สัญญาณสั่นไกเสมอ (ให้เป็นแค่ Pulse)

        // ---------------------------------------------------------------------
        // ระบบตรวจจับการสับสวิตช์เปลี่ยนโหมด On-the-fly (Edge Detection)
        // ถ้าตั้งค่ากล้องเสร็จแล้ว (init_done=1) แต่พบว่าสถานะสวิตช์ปัจจุบัน (sync)
        // ไม่ตรงกับอดีต (colorbar_d) แปลว่าคนดูเพิ่งสับสวิตช์! ให้เริ่มกระบวนการใหม่ทั้งหมด
        // ---------------------------------------------------------------------
        if (init_done && (colorbar_sync != colorbar_d)) begin
            state <= S_RESET;     // กลับไปสถานะเริ่มแรก
            wait_ctr <= 0;
            rom_addr <= 0;
            init_done <= 0;       // ดึงธงลง (สั่งให้วงจรรับภาพหยุดรับภาพชั่วคราว)
            camera_reset <= 1'b0; // ดึงขารีเซ็ตกล้องลงกราวด์ทันที
            active_colorbar <= colorbar_sync; // อัปเดตโหมดใหม่ให้ ROM
            colorbar_d <= colorbar_sync;      // จำสถานะปัจจุบันไว้

        end else case (state)
            // ---------------------------------------------------------------------
```

```
        // S_RESET: ช่วงกดปุ่มรีเซ็ตกล้องแช่ไว้ (Hardware Reset)
        // ----------------------------------------------------------------
        S_RESET: begin
            camera_reset <= 1'b0; // Active-Low: 0 คือการสั่ง Reset กล้อง

            // นับเวลาไปเรื่อยๆ จนกว่าจะครบกำหนด (SETTLE_CYCLES)
            if (wait_ctr < SETTLE_CYCLES) begin
                wait_ctr <= wait_ctr + 1'd1;
            end else begin
                camera_reset <= 1'b1; // ยกเลิกรีเซ็ต (ปล่อยให้กล้องเริ่มทำงาน)
                wait_ctr <= 0;        // เคลียร์ตัวนับเพื่อใช้ในรอบหน้า
                state <= S_WAIT;      // ไปสถานะรอ
            end
        end

        // ----------------------------------------------------------------
        // S_WAIT: ช่วงรอให้กล้องตั้งสติ (Settle Time)
        // ----------------------------------------------------------------
        S_WAIT: begin
            camera_reset <= 1'b1; // รักษาระดับไฟให้กล้องทำงานปกติ

            // แม้จะปลดรีเซ็ตแล้ว แต่ Clock และวงจรในกล้องต้องใช้เวลาตื่นตัว
            // จึงต้องนับเวลาหน่วงแช่ไว้อีก 1 รอบ (SETTLE_CYCLES)
            if (wait_ctr < SETTLE_CYCLES) begin
                wait_ctr <= wait_ctr + 1'd1;
            end else begin
                state <= S_LOAD; // กล้องพร้อมแล้ว ไปโหลดคำสั่งแรกจาก ROM ได้
            end
        end

        // ----------------------------------------------------------------
        // S_LOAD: ดึงคำสั่งจากแผนกคลังข้อมูล (ROM)
        // ----------------------------------------------------------------
        S_LOAD: begin
            // เช็คคำสั่งพิเศษ: FFFF คือรหัสบอกว่า "หมดตารางตั้งค่าแล้ว"
            if (rom_word == 16'hFFFF) begin
                state <= S_DONE; // ข้ามไปจบงานได้เลย

            // เช็คคำสั่งพิเศษ: FFFE คือรหัสบอกว่า "ให้หน่วงเวลาก่อนทำคำสั่งถัดไป"
            // (มักใช้แทรกไว้หลังจากคำสั่ง Software Reset ภายในกล้อง)
            end else if (rom_word == 16'hFFFE) begin
                wait_ctr <= 0;
                state <= S_WAIT; // กลับไปสถานะรอ 1 รอบเวลา
                rom_addr <= rom_addr + 1'd1; // ขยับบรรทัด ROM รอไว้เลย

            // ถ้าเป็นคำสั่งตั้งค่าปกติ
            end else begin
                tx_reg_addr <= rom_word[15:8]; // หั่น 8 บิตบนไปเป็น Register Address
```

```
                tx_reg_data <= rom_word[7:0];  // หั่น 8 บิตล่างไปเป็น Register Data
                state <= S_START; // ขยับไปสั่งลูกน้องคนที่ 1 (SCCB) ทำงาน
            end
        end

        // ----------------------------------------------------------------
        // S_START: สั่นไกสั่งทำงาน (Trigger SCCB)
        // ----------------------------------------------------------------
        S_START: begin
            // ระบบ Handshake: เช็คก่อนว่า SCCB ว่างอยู่ใช่ไหม (!tx_busy)
            if (!tx_busy) begin
                start_tx <= 1'b1; // ปล่อย Pulse สั่งลั่นไก!
                state <= S_NEXT;  // ไปรอรับของ
            end
        end

        // ----------------------------------------------------------------
        // S_NEXT: นั่งรอจนกว่าจะส่งเสร็จ (Wait for Tx Done)
        // ----------------------------------------------------------------
        S_NEXT: begin
            // ถ้า SCCB ส่งสัญญาณ tx_done กลับมา (แปลว่าส่งครบ 27 บิตแล้ว)
            if (tx_done) begin
                rom_addr <= rom_addr + 1'd1; // ขยับไปชี้บรรทัดถัดไปใน ROM
                state <= S_LOAD; // วนกลับไปโหลดคำสั่งบรรทัดต่อไปมาทำ
            end
        end

        // ----------------------------------------------------------------
        // S_DONE: ภารกิจสำเร็จ (Initialization Complete)
        // ----------------------------------------------------------------
        S_DONE: begin
            // ชูธง init_done เป็น 1 ค้างไว้ตลอดกาล
            // (สัญญาณนี้จะวิ่งไปปลดล็อคให้วงจร Capture ภาพเริ่มดึงสีจากกล้องลง BRAM)
            init_done <= 1'b1;
        end
    endcase
  end
end
endmodule
```

Module : ov7670_init_rom

The ov7670_init_rom module acts as a **Read-Only Memory (ROM) lookup table** that stores the specific configuration settings required to initialize the OV7670 camera.

Breakdown of primary functions:

- **Data Storage:** It uses a combinational case statement to map an 8-bit input address to a 16-bit output word. Each 16-bit word consists of an 8-bit camera register address paired with the 8-bit data value to be written to it.
- **Camera Setup:** It holds the exact sequence of register writes needed to configure the camera for **RGB565 video output**.
- **Dynamic Test Mode:** It monitors the colorbar input signal. If active, it alters specific register values (like COM17) on the fly to instruct the camera to output an internal color bar test pattern instead of live video.
- **Controller Guidance:** It outputs special control markers—16'hFFFE to tell the initialization controller to pause and wait for the camera to settle, and 16'hFFFF to signal that the configuration table is complete.

```
// ================================================================================
// โมดูลหน่วยความจำ ROM (Read-Only Memory) สำหรับเก็บตารางตั้งค่ากล้อง OV7670
// เปรียบเสมือน "คู่มือ (Recipe)" ที่บอกว่าต้องไปปรับปุ่มไหนในกล้องให้ออกมาเป็นภาพ RGB565
// ================================================================================
// รูปแบบข้อมูล (16 บิต): {ที่อยู่รีจิสเตอร์ 8 บิต (Register Address), ค่าที่ต้องการตั้ง 8 บิต (Register Value)}
//
// รหัสคำสั่งพิเศษ (Special Markers):
// - 16'hFFFE: สั่งให้ Controller "หยุดรอ (Delay)" เพื่อให้วงจรในกล้องทำงานเสร็จ
// - 16'hFFFF: สั่งให้ Controller "จบการทำงาน (End of Table)" เลิกตั้งค่าได้แล้ว
// ================================================================================
module ov7670_init_rom (
    input  wire [7:0] addr,     // ลำดับบรรทัดที่ Controller ร้องขอมา (เริ่มจาก 00, 01, 02...)
    input  wire       colorbar, // สัญญาณจากสวิตช์ผู้ใช้ ว่าอยากให้กล้องส่งโหมดทดสอบแถบสีหรือไม่
    output reg  [15:0] data     // ข้อมูล 16 บิตที่ส่งกลับไปให้ Controller
);
    // ทำงานทันทีที่มีการร้องขอที่อยู่ (addr) ใหม่ (Combinational Logic)
    always @(*) begin
        case (addr)
            // ================================================================================
            // หมวดที่ 1: การรีเซ็ตกล้อง (System Reset & Settle)
            // ================================================================================
            // 16'h1280 แบ่งเป็น -> Reg 0x12 (COM7), Data 0x80 (สั่งรีเซ็ตทุกค่ากลับเป็นค่าโรงงาน)
            8'h00: data = 16'h1280; // สั่งรีเซ็ตกล้องครั้งที่ 1
            8'h01: data = 16'hFFFE; // สั่งหยุดรอ... ให้ระบบภายในกล้องรีเซ็ตจนเสถียร
            8'h02: data = 16'h1280; // สั่งรีเซ็ตย้ำอีกครั้ง (เพื่อให้ชัวร์ตามคู่มืออ้างอิง)
            8'h03: data = 16'hFFFE; // สั่งหยุดรอ... อีกรอบ

            // ================================================================================
            // หมวดที่ 2: ตั้งค่าฟอร์แมตภาพเป็น RGB565 (Format Setup)
```

```
// ============================================================================
// Reg 0x12 (COM7): เลือกว่าจะใช้ภาพแบบไหน รวมไปถึงเปิด/ปิด Color Bar
// ถ้า colorbar เป็น 1 ส่งค่า 0x06 (RGB + Color Bar), ถ้าเป็น 0 ส่งค่า 0x04 (RGB ปกติ)
8'h04: data = colorbar ? 16'h1206 : 16'h1204;

// ชุดคำสั่งมาตรฐานสำหรับตั้งค่าจังหวะสัญญาณนาฬิกา (Clock) และขอบเขตภาพ
8'h05: data = 16'h1100;
8'h06: data = 16'h0C00;
8'h07: data = 16'h3E00;
8'h08: data = 16'h8C00;
8'h09: data = 16'h0400;

// Reg 0x40 (COM15): ตั้งค่าเป็นโหมด RGB565 (5 บิตแดง, 6 บิตเขียว, 5 บิตน้ำเงิน)
// และกำหนดให้ Output ส่งสัญญาณเต็มสเกล (Full Output Range 00-FF)
8'h0A: data = 16'h40D0;
8'h0B: data = 16'h3A04;

// ============================================================================
// หมวดที่ 3: ระบบชดเชยแสงและปรับแต่งเมทริกซ์สี (AGC & Color Matrix Trims)
// เพื่อแก้ปัญหาเฉพาะของโมดูลกล้องที่ใช้กับ Basys 3 (มักจะติดสีเหลือง/แดง)
// ============================================================================
// Reg 0x14 (COM9): ปรับระบบ Auto Gain Control (AGC)
// เพิ่มความไวแสง (Gain) เพื่อให้ภาพในที่แสงน้อยสว่างขึ้นและใช้งานได้จริง
8'h0C: data = 16'h1438;

// Reg 0x4F, 0x50 (MTX1, MTX2): ค่าสัมประสิทธิ์สมการแปลงสี (Color Matrix Coefficients)
// ถูกปรับแต่งจูนค่ามาเพื่อ "ลดอาการภาพติดสีแดงหรือเหลือง" ที่เป็นจุดอ่อนของกล้องตัวนี้
8'h0D: data = 16'h4F80;
8'h0E: data = 16'h5080;

// ชุดคำสั่งตั้งค่า MTX ตัวอื่นๆ และระบบ DSP ภายใน (Magic numbers มาตรฐาน OV7670)
8'h0F: data = 16'h5100;
8'h10: data = 16'h523D;
8'h11: data = 16'h53A7;
8'h12: data = 16'h54E4;
8'h13: data = 16'h589E;
8'h14: data = 16'h3DC0;
8'h15: data = 16'h1100;
8'h16: data = 16'h1711;
8'h17: data = 16'h1861;
8'h18: data = 16'h32A4;
8'h19: data = 16'h1903;
8'h1A: data = 16'h1A7B;
8'h1B: data = 16'h030A;
8'h1C: data = 16'h0E61;
8'h1D: data = 16'h0F4B;
8'h1E: data = 16'h1602;
8'h1F: data = 16'h1E37;
```

```
8'h20: data = 16'h2102;
8'h21: data = 16'h2291;
8'h22: data = 16'h2907;
8'h23: data = 16'h330B;
8'h24: data = 16'h350B;
8'h25: data = 16'h371D;
8'h26: data = 16'h3871;
8'h27: data = 16'h392A;
8'h28: data = 16'h3C78;
8'h29: data = 16'h4D40;
8'h2A: data = 16'h4E20;
8'h2B: data = 16'h6900;
8'h2C: data = 16'h6B4A;
8'h2D: data = 16'h7410;
8'h2E: data = 16'h8D4F;
8'h2F: data = 16'h8E00;
8'h30: data = 16'h8F00;
8'h31: data = 16'h9000;
8'h32: data = 16'h9100;
8'h33: data = 16'h9600;
8'h34: data = 16'h9A00;
8'h35: data = 16'hB084;

// Reg 0xB1 (ABLC1): เปิดใช้งาน Auto Black-Level Calibration
// ช่วยให้กล้องรู้ว่าระดับความมืดที่แท้จริงคือจุดไหน ภาพจะได้ไม่ฝ้า
8'h36: data = 16'hB18C;
8'h37: data = 16'hB20E;
8'h38: data = 16'hB382;
8'h39: data = 16'hB80A;

// Reg 0x42 (COM17): ตั้งค่า DSP พิเศษ
// เช็คตัวแปร colorbar ถ้าเป็น 1 จะสั่งเปิดโหมดภาพทดสอบ (Color Bars) จากชิป DSP ภายในตัวกล้อง
8'h3A: data = colorbar ? 16'h4208 : 16'h4200;

// ========================================================================
// หมวดที่ 4: การล็อกสมดุลแสงสีขาว (White Balance Lock)
// ========================================================================
// Reg 0x13 (COM8): เปิด Auto Gain / Auto Exposure (AGC/AEC) ให้ปรับแสงตามสภาพจริง
// แต่ **ปิดระบบ Auto White Balance (AWB)** เพื่อไม่ให้สีเปลี่ยนไปมา
8'h3B: data = 16'h13E5;

// Reg 0x41 (COM16): เปิดใช้งาน Color Gain Stage
8'h3C: data = 16'h4108;
8'h3D: data = 16'h6F9F; // AWB Control (ตั้งค่าควบคุม AWB)
8'h3E: data = 16'h6A50; // ตัวช่วยประคองสมดุลช่องสี แดง/เขียว

// เมื่อเราปิด AWB อัตโนมัติไปแล้ว เราต้องใส่ค่าคงที่ (Fixed Gain) ของสีแดงและน้ำเงินเข้าไปเอง
// การทำแบบนี้จะทำให้สีของภาพ "คงที่" ไม่เกิดอาการสีวูบวาบสลับไปมาในแต่ละเฟรม (Frame-to-frame drifting)
```

```
        8'h3F: data = 16'h0180; // Reg 0x01: อัตราขยายสีน้ำเงิน (Blue Gain) ค่า Default คือ 0x80
        8'h40: data = 16'h0250; // Reg 0x02: อัตราขยายสีแดง (Red Gain) ปรับลดลงมาเหลือ 0x50 (เพราะกล้องนี้มักจะอมแดง)

        // ========================================================================
        // หมวดที่ 5: สิ้นสุดตาราง (End of Table)
        // ========================================================================
        // เมื่อ Controller อ่านเจอรหัสนี้ จะหยุดการส่ง SCCB และประกาศว่ากล้องพร้อมใช้งาน
        8'h41: data = 16'hFFFF;

        // กรณีมีคนอ่านเกินบรรทัดที่มีอยู่ ให้ส่งรหัสจบกลับไปเสมอ ป้องกันระบบแฮงค์
        default: data = 16'hFFFF;
    endcase
  end
endmodule
```

Module : sccb_write_byte

The sccb_write_byte module acts as a **hardware serialization engine** for the Serial Camera Control Bus (SCCB) protocol, which is the I2C-like interface required to configure Omnivision cameras.

- **Protocol Serialization:** When trigger **start** signal, it takes three 8-bit inputs (the camera's device address, the target register address, and the data value) and serializes them into a continuous 27-bit.
- **Clock Generation:** It divides the high-speed system clock (100 MHz) down to a lower frequency (25 kHz) suitable for the camera interface, outputting this as the serial clock (sioc).
- **Open-Drain Signaling:** It mimics an open-drain output by only pulling the **siod** line low (0) and releasing it (to high-impedance Z) so an external pull-up resistor can pull it to logic
- **State Management:** A state machine handles the exact timing required by the SCCB protocol, including generating the START condition, managing the 9th bit (don't-care/ACK) cycles after every byte, and executing the final STOP condition.
- **Handshaking:** It outputs **busy** and **done** signals to tell the controller (ov7670_init_controller) when it is safe to send the next register command.

```
// ================================================================================
// โมดูลส่งข้อมูลผ่านโปรโตคอล SCCB (คล้าย I2C) สำหรับตั้งค่ากล้อง OV7670
//
// หน้าที่หลัก: รับคำสั่ง 3 ไบต์ (Device Address, Register Address, Data)
// มาแปลงเป็นสัญญาณอนุกรม (Serial) ส่งออกไปทีละบิตตามมาตรฐาน SCCB
// ================================================================================
module sccb_write_byte #(
    // ตัวหารความถี่ (Clock Divider) เพื่อสร้างจังหวะการส่งข้อมูล
    // บอร์ดทำงานที่ 100 MHz หารด้วย 2000 จะได้จังหวะการทำงานประมาณ 50 kHz
    // (ความถี่จริงของ SIOC จะช้ากว่านี้เพราะใช้หลาย Tick ต่อ 1 บิต)
    parameter integer DIV = 2000
) (
    input  wire clk,           // สัญญาณนาฬิกาหลักของระบบ (100 MHz)
    input  wire reset,         // สัญญาณรีเซ็ตวงจร
    input  wire start,         // สัญญาณกระตุ้นให้เริ่มส่งข้อมูล (Pulse 1 clock)
    input  wire [7:0] dev_addr, // ที่อยู่ของตัวกล้อง (มักจะเป็น 8'h42 สำหรับเขียน)
    input  wire [7:0] reg_addr, // ตำแหน่ง Register ภายในกล้องที่ต้องการตั้งค่า
    input  wire [7:0] reg_data, // ข้อมูลที่ต้องการเขียนลงไปใน Register นั้น
    output wire sioc,          // สัญญาณนาฬิกา SCCB (Serial Clock) ส่งไปให้กล้อง
    inout  wire siod,          // สัญญาณข้อมูล SCCB (Serial Data) แบบ 2 ทิศทาง
    output reg  busy = 1'b0,    // สถานะบอกว่า "กำลังส่งข้อมูลอยู่ ห้ามกวน"
    output reg  done = 1'b0     // สถานะบอกว่า "ส่งข้อมูลเสร็จแล้ว 1 ชุด"
);

    // ================================================================================
    // ส่วนที่ 1: ตัวกำเนิดจังหวะ (Baud Rate / Tick Generator)
    // ================================================================================
    reg [15:0] divcnt = 0; // ตัวนับสำหรับหารความถี่
    reg scl = 1'b1;        // ตัวแปรจำลองสถานะของสาย SIOC
    reg tick = 1'b0;       // สัญญาณชีพจร (Pulse) ที่จะดังขึ้น 1 ครั้งเมื่อนับครบ
```

```
// สร้างสัญญาณ tick เป็นจังหวะอ้างอิงให้ State Machine ทำงาน
always @(posedge clk) begin
    if (reset) begin
        divcnt <= 0;
        tick <= 0;
    end else begin
        if (divcnt == DIV-1) begin
            divcnt <= 0;
            tick <= 1'b1; // ส่ง Tick ออกมา 1 จังหวะ
        end else begin
            divcnt <= divcnt + 1'd1;
            tick <= 1'b0;
        end
    end
end

// ============================================================================
// ส่วนที่ 2: วงจรควบคุมสายส่งข้อมูลทางกายภาพ (Physical Layer / Open-Drain)
// ============================================================================
reg sda_out = 1'b1; // ข้อมูลที่เราต้องการส่งออกไป (0 หรือ 1)
reg sda_oe  = 1'b1; // Output Enable (อนุญาตให้ FPGA ดึงสายไฟหรือไม่)

// เชื่อม SCL เข้ากับสาย SIOC โดยตรง
assign sioc = scl;

// กฎของ Open-Drain (เช่นเดียวกับ I2C):
// - ถ้าอยากส่ง '0' ให้ FPGA ดึงสายลงกราวด์ (1'b0)
// - ถ้าอยากส่ง '1' ให้ FPGA ปล่อยสายว่าง (1'bz หรือ High-Impedance)
//   แล้วตัวต้านทาน Pull-up บนบอร์ดจะดึงไฟขึ้นเป็น '1' เอง ป้องกันการลัดวงจร
assign siod = (sda_oe && !sda_out) ? 1'b0 : 1'bz;

// ============================================================================
// ส่วนที่ 3: โครงสร้างข้อมูล (Shift Register & Bit Counter)
// ============================================================================
reg [5:0] bitcnt = 0;   // ตัวนับว่าตอนนี้ส่งไปแล้วกี่บิต
reg [26:0] shreg = 27'h0; // ท่อพักข้อมูล (Shift Register) ขนาด 27 บิต

// ประกาศชื่อสถานะ (State) ต่างๆ ของ FSM เพื่อให้อ่านโค้ดง่ายขึ้น
localparam I_IDLE  = 3'd0; // รอคำสั่ง
localparam I_START = 3'd1; // สร้างจังหวะเริ่ม (Start Condition)
localparam I_BITL  = 3'd2; // จังหวะวางข้อมูล (Clock Low)
localparam I_BITH  = 3'd3; // จังหวะให้กล้องอ่านข้อมูล (Clock High)
localparam I_STOP0 = 3'd4; // เตรียมสร้างจังหวะจบ
localparam I_STOP1 = 3'd5; // สร้างจังหวะจบ (Stop Condition) ขั้น 1
localparam I_STOP2 = 3'd6; // สร้างจังหวะจบ ขั้น 2

reg [2:0] state = I_IDLE;
```

```
// ============================================================================
// ส่วนที่ 4: เครื่องจักรสถานะ (Finite State Machine - FSM)
// ============================================================================
always @(posedge clk) begin
    if (reset) begin
        state <= I_IDLE;
        busy <= 1'b0;
        done <= 1'b0;
        scl <= 1'b1;
        sda_out <= 1'b1;
        sda_oe <= 1'b1;
        bitcnt <= 0;
    end else begin
        done <= 1'b0; // เคลียร์สัญญาณ Done เสมอ (ให้เป็นแค่ Pulse สั้นๆ)

        // ถ้าเครื่องว่างอยู่ และมีคนสั่ง start
        if ((state == I_IDLE) && start) begin
            scl <= 1'b1;
            sda_out <= 1'b1;
            sda_oe <= 1'b1;

            // จัดเรียงแพ็กเกจข้อมูล 27 บิต:
            // [Device Addr 8 บิต] + [Don't Care 1 บิต] +
            // [Reg Addr 8 บิต] + [Don't Care 1 บิต] +
            // [Data 8 บิต] + [Don't Care 1 บิต]
            // (บิต '1' ที่คั่นอยู่คือบิตเว้นว่างให้กล้องส่ง Ack กลับมา แต่เราไม่ได้อ่าน)
            shreg <= {dev_addr, 1'b1, reg_addr, 1'b1, reg_data, 1'b1};

            bitcnt <= 6'd26; // นับถอยหลัง 27 บิต (26 ถึง 0)
            busy <= 1'b1;    // ประกาศว่าเครื่องไม่ว่างแล้ว
            state <= I_START; // ขยับไปสถานะเริ่มต้น

        // State Machine จะขยับก็ต่อเมื่อมีสัญญาณ tick เข้ามาเท่านั้น
        end else if (tick) begin
            case (state)
                I_IDLE: begin
                    // อยู่เฉยๆ รักษาให้สาย Clock และ Data เป็น High (สภาวะปกติ)
                    scl <= 1'b1;
                    sda_out <= 1'b1;
                    sda_oe <= 1'b1;
                    busy <= 1'b0;
                end

                I_START: begin
                    // เงื่อนไข Start Condition ของ SCCB:
                    // ขณะที่สาย Clock (scl) เป็น High ให้ดึงสาย Data (sda) ลงมาเป็น Low
                    scl <= 1'b1;
```

```
        sda_out <= 1'b0;
        state <= I_BITL;
    end

    I_BITL: begin
        scl <= 1'b0; // ดึง Clock ลง Low เพื่อให้กล้องรู้ว่า "หยุดอ่านก่อน เรากำลังเปลี่ยนข้อมูล"

        // ปลดการเชื่อมต่อ (High-Z) ในบิตที่ 9 (Don't care/Ack bit)
        // ตำแหน่งที่ 18, 9, 0 คือบิตคั่นระหว่างแต่ละไบต์ตามกฎ SCCB
        sda_oe <= !((bitcnt == 6'd18) || (bitcnt == 6'd9) || (bitcnt == 6'd0));

        // หยิบบิตซ้ายสุดของ Shift Register มาวางบนสาย Data
        sda_out <= shreg[26];
        state <= I_BITH;
    end

    I_BITH: begin
        scl <= 1'b1; // ดัน Clock ขึ้น High เพื่อสั่งกล้องว่า "อ่านข้อมูลบนสายได้เลย!"

        // เลื่อนบิตใน Shift Register ไปทางซ้าย 1 ตำแหน่ง เพื่อเตรียมข้อมูลบิตถัดไป
        shreg <= {shreg[25:0],1'b1};

        if (bitcnt == 0)
            state <= I_STOP0; // ถ้าส่งครบ 27 บิตแล้ว ให้ไปสร้าง Stop Condition
        else begin
            bitcnt <= bitcnt - 1'd1; // ลดจำนวนบิตที่เหลือลง
            state <= I_BITL; // วนกลับไปสถานะ Clock Low เพื่อส่งบิตถัดไป
        end
    end

    // ขั้นตอนการสร้าง Stop Condition: Data ต้องเปลี่ยนจาก Low เป็น High ในขณะที่ Clock เป็น High
    I_STOP0: begin
        sda_oe <= 1'b1;
        scl <= 1'b0;      // ดึง Clock ลง Low ก่อน
        sda_out <= 1'b0;  // ดึง Data ลง Low เตรียมไว้
        state <= I_STOP1;
    end

    I_STOP1: begin
        sda_oe <= 1'b1;
        scl <= 1'b1;      // ดัน Clock ขึ้น High
        sda_out <= 1'b0;  // Data ยังคงเป็น Low อยู่
        state <= I_STOP2;
    end

    I_STOP2: begin
        scl <= 1'b1;      // Clock ยังเป็น High
        sda_out <= 1'b1;  // ปล่อย Data ให้ลอยขึ้น High (เสร็จสมบูรณ์ Stop Condition)
```

```
                busy <= 1'b0;     // เครื่องว่างแล้ว
                done <= 1'b1;     // ส่ง Pulse ว่า "จบงาน" ให้ Controller ตัวแม่รู้
                state <= I_IDLE;  // กลับไปรอคำสั่งใหม่
            end
        endcase
    end
  end
 end
endmodule
```

Module : vga_640x480

The vga_640x480 module acts as a **hardware display timing generator**. It establishes the coordinate system and synchronization signals needed to display a 640x480 pixel image at a 60 Hz refresh rate on a VGA monitor.

- **Pixel Coordinate Generation:** It maintains two counters, sx (horizontal) and sy (vertical), which increment every 25 MHz pixel clock cycle. These counters sweep across the entire screen area, including the visible pixels and the unseen blanking intervals required by the VGA standard.
- **Sync Signal Generation:** It produces active-low **hsync** (horizontal sync) and **vsync** (vertical sync) pulses at precise pixel coordinate ranges (sx between 656 and 751). These pulses tell the monitor when to move to the next row and when to start a new frame.
- **Video Qualification:** It outputs an active_video signal, which is strictly high (true) only when the counters are within the 640x480 visible screen area. The main controller uses this signal to ensure pixel data is only sent to the monitor during visible periods.

```
`timescale 1ns / 1ps

// ============================================================================
// เครื่องกำเนิดสัญญาณ VGA ความละเอียด 640x480 ที่อัตรารีเฟรช 60 Hz
// ทำหน้าที่สร้างพิกัดพิกเซล (X,Y) และสัญญาณควบคุมจังหวะหน้าจอ (Hsync, Vsync)
// ============================================================================
module vga_640x480 (
    // สัญญาณนาฬิกาหลัก ต้องเป็น 25 MHz เท่านั้นสำหรับความละเอียด 640x480 @ 60Hz
    input  wire       clk25,

    // พิกัดหัวอ่านปัจจุบันบนหน้าจอ
    output reg [9:0]  sx = 0, // แกน X (แนวนอน) นับตั้งแต่ 0 - 799
    output reg [9:0]  sy = 0, // แกน Y (แนวตั้ง) นับตั้งแต่ 0 - 524

    // สัญญาณควบคุมจอภาพ (มาตรฐาน VGA ใช้ Active-Low คือเป็น 0 เมื่อสั่งการ)
    output wire       hsync,  // สั่งให้หัวอ่านปัดกลับไปเริ่มบรรทัดใหม่ทางซ้าย
    output wire       vsync,  // สั่งให้หัวอ่านปัดกลับไปเริ่มเฟรมใหม่ที่มุมซ้ายบน

    // สัญญาณรับรองพื้นที่แสดงผล
    output wire       active_video, // เป็น 1 เมื่อหัวอ่านอยู่ในพื้นที่ที่อนุญาตให้ส่งสีออกไปได้
    output wire       frame_start   // ส่งสัญญาณ Pulse สั้นๆ 1 จังหวะ เมื่อเริ่มวาดเฟรมใหม่
);

    // ============================================================================
    // สเปคมาตรฐานเวลาของ VGA 640x480 @ 60 Hz (ใช้ Clock 25 MHz)
    // ----------------------------------------------------------------------------
    // แกนนอน (Horizontal): วาดภาพจริง 640, เว้นหน้า(FP) 16, สั่งซิงค์ 96, เว้นหลัง(BP) 48
    // รวม 1 บรรทัดต้องนับให้ครบ = 800 จังหวะ (Clock)
    //
    // แกนตั้ง (Vertical): วาดภาพจริง 480, เว้นหน้า(FP) 10, สั่งซิงค์ 2, เว้นหลัง(BP) 33
    // รวม 1 หน้าจอต้องกวาดให้ครบ = 525 บรรทัด
```

```
    // ============================================================================

    // ============================================================================
    // ส่วนที่ 1: วงจรนับตำแหน่งพิกัด (Coordinate Counters)
    // ============================================================================
    // ทำงานทุกๆ ขอบขาขึ้นของ Clock 25 MHz (1 Clock = ขยับหัวอ่านไปทางขวา 1 พิกเซล)
    always @(posedge clk25) begin

        // ถ้าหัวอ่านวิ่งไปจนสุดขอบแนวนอนของเส้นทาง (พิกเซลที่ 799)
        if (sx == 799) begin
            sx <= 0; // ดึงหัวอ่านกลับมาที่พิกัด 0 ทางซ้ายสุด

            // เมื่อจบบรรทัด ให้เช็คว่ากวาดครบทุกบรรทัดในแนวตั้งหรือยัง (บรรทัดที่ 524)
            if (sy == 524)
                sy <= 0; // ถ้าครบแล้ว ให้ดึงหัวอ่านกลับไปบรรทัดบนสุด (เริ่มเฟรมใหม่)
            else
                sy <= sy + 1; // ถ้ายังไม่ครบ ให้ปืดลงมาวาดบรรทัดถัดไป

        // ถ้ายังไม่สุดขอบแนวนอน
        end else begin
            sx <= sx + 1; // เลื่อนหัวอ่านไปทางขวาเรื่อยๆ ทีละ 1 พิกเซล
        end
    end
    // ============================================================================
    // ส่วนที่ 2: สร้างสัญญาณควบคุมจอ (Sync Generation)
    // ============================================================================
    // สัญญาณ HSYNC (แนวนอน): ทำงานเป็น Active-Low (ใส่ ~ เพื่อกลับบิต)
    // จอจะรับรู้คำสั่ง HSYNC ในช่วงพิกัด: วาดภาพ(640) + หน้า(16) = เริ่มที่พิกัด 656
    // ความกว้างของคำสั่งคือ 96 จังหวะ ดังนั้นจะสิ้นสุดที่: 656 + 96 = พิกัด 752
    assign hsync = ~((sx >= 656) && (sx < 752));

    // สัญญาณ VSYNC (แนวตั้ง): ทำงานเป็น Active-Low (ใส่ ~ เพื่อกลับบิต)
    // จอจะรับรู้คำสั่ง VSYNC ในช่วงบรรทัด: วาดภาพ(480) + หน้า(10) = เริ่มที่บรรทัด 490
    // ความกว้างของคำสั่งคือ 2 บรรทัด ดังนั้นจะสิ้นสุดที่: 490 + 2 = บรรทัด 492
    assign vsync = ~((sy >= 490) && (sy < 492));

    // ============================================================================
    // ส่วนที่ 3: ระบุพื้นที่วาดภาพ (Blanking Qualification)
    // ============================================================================
    // active_video จะเป็น 1 ก็ต่อเมื่อหัวอ่านอยู่ในพื้นที่ 640x480 เท่านั้น
    // หากเกินกว่านี้ (อยู่ในช่วง Porch หรือ Sync) จะเป็น 0
    // (ระบบข้างนอกต้องนำค่านี้ไปตัดให้สี RGB เป็นดำ (0) ในช่วงนี้ ไม่เช่นนั้นจอจะรวน)
    assign active_video = (sx < 640) && (sy < 480);

    // สร้างสัญญาณเตือนวงจรภายนอกว่า "กำลังเริ่มวาดภาพแผ่นใหม่แล้วนะ"
    assign frame_start  = (sx == 0) && (sy == 0);

endmodule
```

Constraint : basys3_ov7670_fifo.xdc

This basys3_ov7670_fifo.xdc file serves as the physical blueprint for the project, mapping your Verilog logic to the Basys3 hardware.

- **Physical Pin Mapping:** Bridges abstract Verilog ports (e.g., vgaRed, cam_d) to specific electrical pins on the FPGA chip (e.g., G19, P17).
- **Timing Definition:** Establishes the **100 MHz master clock** constraint, ensuring all internal logic and state machines operate within precise 10ns intervals.
- **Camera Integration:** Configures the **OV7670 interface**, including data lines, synchronization signals (vsync, href), and the bidirectional SCCB (I2C-like) bus with internal pull-up resistors.
- **Display Synchronization:** Routes the **VGA signals** (Hsync, Vsync, and 12-bit RGB) to the onboard DAC to drive an external monitor at 640x480 resolution.
- **Hardware Overrides:** Includes critical routing instructions (CLOCK_DEDICATED_ROUTE) to allow the use of non-standard pins for high-speed signals like the camera pixel clock.
- **User I/O:** Links physical **switches and LEDs** to the logic, enabling real-time control over image filters and system status monitoring.

```
## 100 MHz board clock
set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports CLK100MHZ]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLK100MHZ]
## Debug switches
## sw[0] = VGA internal color bars
## sw[1] = live camera signal viewer
## sw[4:2] = filter mode: 0 normal, 1 grayscale, 2 invert, 3 red, 4 green, 5 blue
## sw[5] = OV7670 internal color bars; toggling this re-runs camera configuration
## sw[7:6] = unused debug switches
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports {sw[0]}]
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports {sw[2]}]
set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
set_property -dict { PACKAGE_PIN W15 IOSTANDARD LVCMOS33 } [get_ports {sw[4]}]
set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports {sw[5]}]
set_property -dict { PACKAGE_PIN W14 IOSTANDARD LVCMOS33 } [get_ports {sw[6]}]
set_property -dict { PACKAGE_PIN W13 IOSTANDARD LVCMOS33 } [get_ports {sw[7]}]
## LEDs
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports {led[0]}]
set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports {led[1]}]
set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports {led[2]}]
set_property -dict { PACKAGE_PIN V19 IOSTANDARD LVCMOS33 } [get_ports {led[3]}]
set_property -dict { PACKAGE_PIN W18 IOSTANDARD LVCMOS33 } [get_ports {led[4]}]
set_property -dict { PACKAGE_PIN U15 IOSTANDARD LVCMOS33 } [get_ports {led[5]}]
set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVCMOS33 } [get_ports {led[6]}]
set_property -dict { PACKAGE_PIN V14 IOSTANDARD LVCMOS33 } [get_ports {led[7]}]
set_property -dict { PACKAGE_PIN V13 IOSTANDARD LVCMOS33 } [get_ports {led[8]}]
set_property -dict { PACKAGE_PIN V3  IOSTANDARD LVCMOS33 } [get_ports {led[9]}]
```

```
set_property -dict { PACKAGE_PIN W3  IOSTANDARD LVCMOS33 } [get_ports {led[10]}]
set_property -dict { PACKAGE_PIN U3  IOSTANDARD LVCMOS33 } [get_ports {led[11]}]
set_property -dict { PACKAGE_PIN P3  IOSTANDARD LVCMOS33 } [get_ports {led[12]}]
set_property -dict { PACKAGE_PIN N3  IOSTANDARD LVCMOS33 } [get_ports {led[13]}]
set_property -dict { PACKAGE_PIN P1  IOSTANDARD LVCMOS33 } [get_ports {led[14]}]
set_property -dict { PACKAGE_PIN L1  IOSTANDARD LVCMOS33 } [get_ports {led[15]}]
## OV7670 camera pins from the project wiring table
set_property -dict { PACKAGE_PIN P17 IOSTANDARD LVCMOS33 } [get_ports {cam_d[0]}]
set_property -dict { PACKAGE_PIN N17 IOSTANDARD LVCMOS33 } [get_ports {cam_d[1]}]
set_property -dict { PACKAGE_PIN M19 IOSTANDARD LVCMOS33 } [get_ports {cam_d[2]}]
set_property -dict { PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports {cam_d[3]}]
set_property -dict { PACKAGE_PIN L17 IOSTANDARD LVCMOS33 } [get_ports {cam_d[4]}]
set_property -dict { PACKAGE_PIN K17 IOSTANDARD LVCMOS33 } [get_ports {cam_d[5]}]
set_property -dict { PACKAGE_PIN C16 IOSTANDARD LVCMOS33 } [get_ports {cam_d[6]}]
set_property -dict { PACKAGE_PIN B16 IOSTANDARD LVCMOS33 } [get_ports {cam_d[7]}]
set_property -dict { PACKAGE_PIN A17 IOSTANDARD LVCMOS33 } [get_ports cam_href]
set_property -dict { PACKAGE_PIN A16 IOSTANDARD LVCMOS33 } [get_ports cam_pclk_buf]
set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports cam_pwdn]
set_property -dict { PACKAGE_PIN P18 IOSTANDARD LVCMOS33 } [get_ports cam_reset]
set_property -dict { PACKAGE_PIN A14 IOSTANDARD LVCMOS33 } [get_ports cam_sioc]
set_property -dict { PACKAGE_PIN A15 IOSTANDARD LVCMOS33 } [get_ports cam_siod]
set_property PULLUP true [get_ports cam_siod]
set_property -dict { PACKAGE_PIN B15 IOSTANDARD LVCMOS33 } [get_ports cam_vsync]
set_property -dict { PACKAGE_PIN C15 IOSTANDARD LVCMOS33 } [get_ports cam_xclk]
## Camera PCLK is wired to a non-clock-capable pin in this pinout.
## This lets Vivado route that input to the inferred clock buffer.
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets -of_objects [get_ports cam_pclk_buf]]
## VGA
set_property -dict { PACKAGE_PIN G19 IOSTANDARD LVCMOS33 } [get_ports {vgaRed[0]}]
set_property -dict { PACKAGE_PIN H19 IOSTANDARD LVCMOS33 } [get_ports {vgaRed[1]}]
set_property -dict { PACKAGE_PIN J19 IOSTANDARD LVCMOS33 } [get_ports {vgaRed[2]}]
set_property -dict { PACKAGE_PIN N19 IOSTANDARD LVCMOS33 } [get_ports {vgaRed[3]}]
set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVCMOS33 } [get_ports {vgaGreen[0]}]
set_property -dict { PACKAGE_PIN H17 IOSTANDARD LVCMOS33 } [get_ports {vgaGreen[1]}]
set_property -dict { PACKAGE_PIN G17 IOSTANDARD LVCMOS33 } [get_ports {vgaGreen[2]}]
set_property -dict { PACKAGE_PIN D17 IOSTANDARD LVCMOS33 } [get_ports {vgaGreen[3]}]
set_property -dict { PACKAGE_PIN N18 IOSTANDARD LVCMOS33 } [get_ports {vgaBlue[0]}]
set_property -dict { PACKAGE_PIN L18 IOSTANDARD LVCMOS33 } [get_ports {vgaBlue[1]}]
set_property -dict { PACKAGE_PIN K18 IOSTANDARD LVCMOS33 } [get_ports {vgaBlue[2]}]
set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports {vgaBlue[3]}]

set_property -dict { PACKAGE_PIN P19 IOSTANDARD LVCMOS33 } [get_ports Hsync]
set_property -dict { PACKAGE_PIN R19 IOSTANDARD LVCMOS33 } [get_ports Vsync]
```

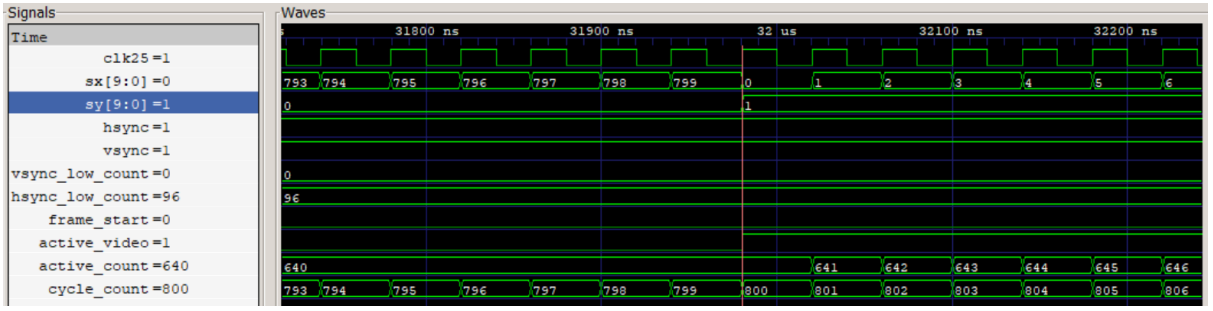
Testbench

tb_vga_640x480 : Test VGA timing

- ➢ clk25 toggles every 20 ns half-period
- ➢ sx counts from 0 to 799
- ➢ sy increments once every time sx wraps
- ➢ sy counts from 0 to 524
- ➢ active_video is high only when sx < 640 and sy < 480
- ➢ hsync goes low from sx = 656 to sx = 751
- ➢ vsync goes low from sy = 490 to sy = 491
- ➢ frame_start pulses when sx = 0 and sy = 0

What it should look like:
One line = 800 clk25 cycles
One frame = 800 * 525 cycles
Visible region = 640 * 480 cycles
HSYNC is active-low
VSYNC is active-low

Result: PASS: VGA timing generated one 640x480 frame correctly.




```
`timescale 1ns / 1ps

// Test one complete 640x480 VGA frame. It counts active-video pixels and the
// active-low HSYNC/VSYNC pulse widths against the expected timing constants.
module tb_vga_640x480;
    reg clk25 = 1'b0;
    wire [9:0] sx;
    wire [9:0] sy;
    wire hsync;
```

```
wire vsync;
wire active_video;
wire frame_start;

integer active_count = 0;
integer hsync_low_count = 0;
integer vsync_low_count = 0;
integer cycle_count = 0;

always #20 clk25 = ~clk25; // 25 MHz

vga_640x480 dut (
    .clk25(clk25),
    .sx(sx),
    .sy(sy),
    .hsync(hsync),
    .vsync(vsync),
    .active_video(active_video),
    .frame_start(frame_start)
);

initial begin
    $dumpfile("tb_vga_640x480.vcd");
    $dumpvars(0, tb_vga_640x480);

    repeat (800 * 525) begin
        @(posedge clk25);
        cycle_count = cycle_count + 1;
        if (active_video) active_count = active_count + 1;
        if (!hsync) hsync_low_count = hsync_low_count + 1;
        if (!vsync) vsync_low_count = vsync_low_count + 1;
    end

    if (active_count != 640 * 480) begin
        $display("FAIL: active video count = %0d", active_count);
        $finish;
    end

    if (hsync_low_count != 96 * 525) begin
        $display("FAIL: HSYNC low count = %0d", hsync_low_count);
        $finish;
    end

    if (vsync_low_count != 2 * 800) begin
        $display("FAIL: VSYNC low count = %0d", vsync_low_count);
        $finish;
    end
```

```
        $display("PASS: VGA timing generated one 640x480 frame correctly.");
        $finish;
    end
endmodule
```

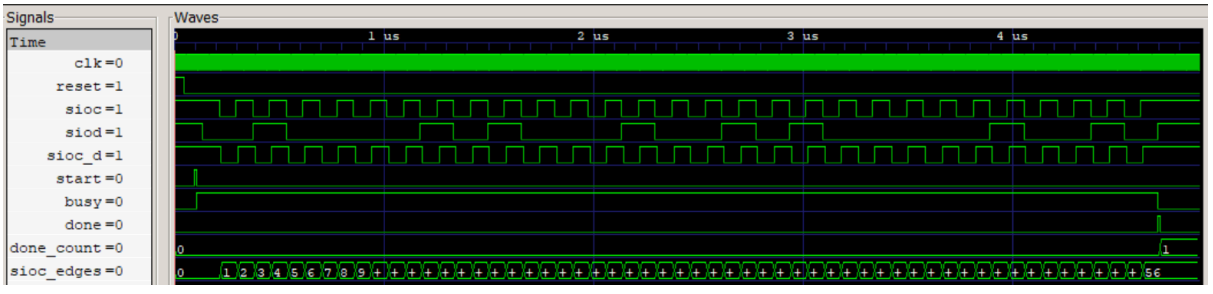
---

tb_sccb_write_byte : Test SCCB (Serial Camera Control Bus) transaction

- ➢ reset starts high, then goes low
- ➢ start pulses high for one clk cycle
- ➢ busy goes high during the transaction
- ➢ sioc begins toggling
- ➢ siod changes while sioc is low
- ➢ siod is stable while sioc is high
- ➢ done pulses once at the end
- ➢ busy returns low
- ➢ sioc returns high
- ➢ siod returns high

What it should look like:
START
0x42 device write address
ACK/released bit
0x12 register address
ACK/released bit
0x04 register data
ACK/released bit
STOP

Result: PASS: SCCB write produced one transaction and returned to idle.



```
`timescale 1ns / 1ps

// Test one SCCB register-write transaction. The test checks that the write
// engine pulses done once, returns the bus to idle, and produces SIOC activity.
module tb_sccb_write_byte;
    reg clk = 1'b0;
    reg reset = 1'b1;
    reg start = 1'b0;
    wire sioc;
    tri1 siod;
    wire busy;
    wire done;

    integer done_count = 0;
    integer sioc_edges = 0;
    reg sioc_d = 1'b1;

    always #5 clk = ~clk; // 100 MHz

    sccb_write_byte #(
        .DIV(8)
    ) dut (
        .clk(clk),
        .reset(reset),
        .start(start),
        .dev_addr(8'h42),
        .reg_addr(8'h12),
        .reg_data(8'h04),
        .sioc(sioc),
        .siod(siod),
        .busy(busy),
        .done(done)
    );

    always @(posedge clk) begin
        sioc_d <= sioc;
        if (sioc_d != sioc) sioc_edges = sioc_edges + 1;
        if (done) done_count = done_count + 1;
    end

    initial begin
        $dumpfile("tb_sccb_write_byte.vcd");
        $dumpvars(0, tb_sccb_write_byte);

        repeat (5) @(posedge clk);
        reset <= 1'b0;
        repeat (5) @(posedge clk);
```

```
        start <= 1'b1;
        @(posedge clk);
        start <= 1'b0;

        wait (done == 1'b1);
        repeat (20) @(posedge clk);

        if (done_count != 1) begin
            $display("FAIL: done_count = %0d", done_count);
            $finish;
        end

        if (busy != 1'b0 || sioc != 1'b1 || siod != 1'b1) begin
            $display("FAIL: bus did not return to idle. busy=%b sioc=%b siod=%b", busy, sioc, siod);
            $finish;
        end

        if (sioc_edges < 50) begin
            $display("FAIL: too few SIOC transitions: %0d", sioc_edges);
            $finish;
        end

        $display("PASS: SCCB write produced one transaction and returned to idle.");
        $finish;
    end
endmodule
```

---

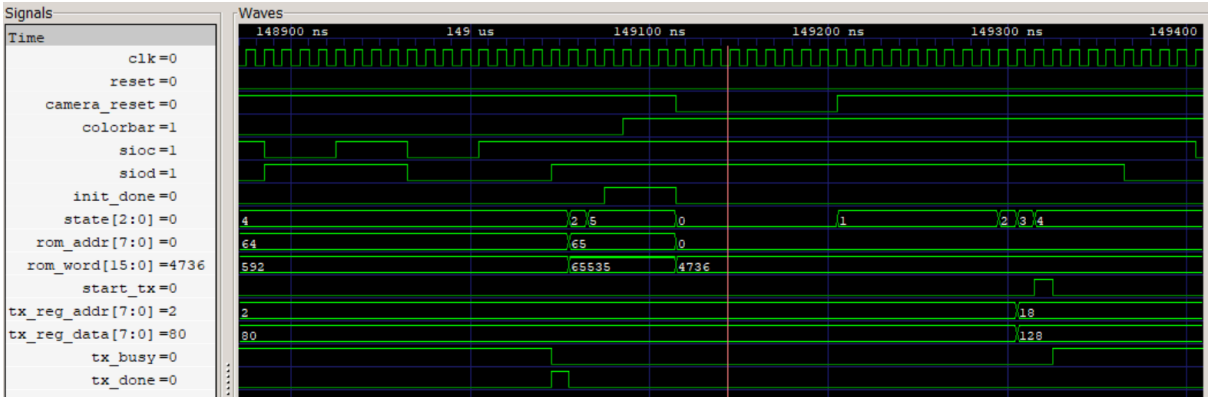
tb_ov7670_init_controller : Test camera register initialization

- ➢ reset starts high
- ➢ camera_reset is held low
- ➢ controller waits
- ➢ camera_reset goes high
- ➢ controller reads ROM entries
- ➢ start_tx pulses for each real register write
- ➢ sccb busy goes high for each SCCB transaction
- ➢ tx_done pulses after each transaction
- ➢ rom_addr increments through the register table
- ➢ init_done goes high at the end
- ➢ colorbar toggles
- ➢ init_done goes low again

- ➢ initialization sequence repeats
- ➢ init_done goes high again

When rom_word is 16'hFFFE, expect the controller to wait instead of starting an SCCB write.
When rom_word is 16'hFFFF, expect the controller to finish and assert "init_done = 1".

Result: PASS: OV7670 init sequence completed and re-ran after colorbar toggle.




```
`timescale 1ns / 1ps

// Test the OV7670 initialization sequencer. The settle time and SCCB divider
// are reduced for simulation, but the state-machine behavior is the same as
// hardware: reset, wait, write the ROM table, and re-run on colorbar toggle.
module tb_ov7670_init_controller;
    reg clk = 1'b0;
    reg reset = 1'b1;
    reg colorbar = 1'b0;
    wire camera_reset;
    wire sioc;
    tri1 siod;
    wire init_done;

    integer cycles = 0;

    always #5 clk = ~clk; // 100 MHz

    ov7670_init_controller #(
        .SETTLE_CYCLES(8),
        .SCCB_DIV(4)
    ) dut (
        .clk(clk),
```

```
        .reset(reset),
        .colorbar(colorbar),
        .camera_reset(camera_reset),
        .sioc(sioc),
        .siod(siod),
        .init_done(init_done)
    );

    initial begin
        $dumpfile("tb_ov7670_init_controller.vcd");
        $dumpvars(0, tb_ov7670_init_controller);

        repeat (4) @(posedge clk);
        reset <= 1'b0;

        while (!init_done && cycles < 200000) begin
            @(posedge clk);
            cycles = cycles + 1;
        end

        if (!init_done) begin
            $display("FAIL: init_done was not asserted.");
            $finish;
        end

        if (camera_reset != 1'b1) begin
            $display("FAIL: camera_reset was not released.");
            $finish;
        end

        colorbar <= 1'b1;
        @(posedge clk);
        wait (init_done == 1'b0);
        wait (init_done == 1'b1);

        $display("PASS: OV7670 init sequence completed and re-ran after colorbar toggle.");
        $finish;
    end
endmodule
```

---

tb_top_capture_filters : Test camera pixel capture and framebuffer storage (This test bypass camera initialization by forcing dut.cam_init_done = 1)

➢ The test sends four RGB565 pixels:

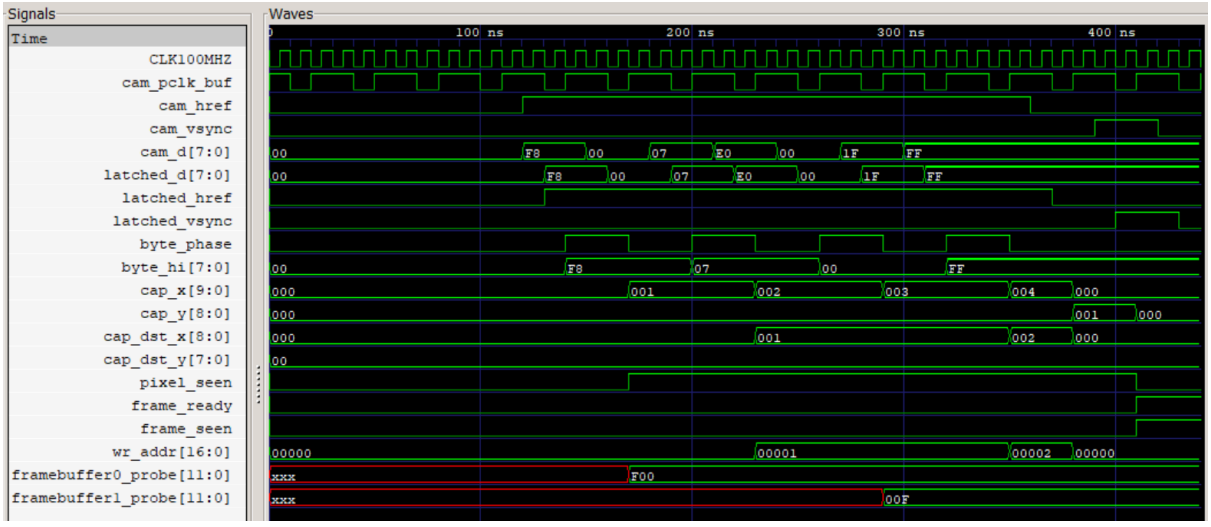
- F8 00 = red -> should be stored
- 07 E0 = green -> should be skipped
- 00 1F = blue -> should be stored
- FF FF = white -> should be skipped

➢ Green and white are skipped because the design downsamples horizontally by 2 and stores only even source pixels

- framebuffer[0] = 12'hF00
- framebuffer[1] = 12'h00F

➢ byte_phase toggles 0 -> 1 -> 0 for each two-byte RGB565 pixel
➢ byte_hi stores the first byte of each pixel
➢ cap_x increments once per full pixel
➢ framebuffer[0] becomes F00 after red
➢ green is skipped
➢ framebuffer[1] becomes 00F after blue
➢ white is skipped
➢ VSYNC marks a frame boundary
➢ frame_ready becomes 1 after VSYNC

Note: framebuffer0_probe = dut.framebuffer[0] and framebuffer1_probe = dut.framebuffer[1]
Result: PASS: top-level capture wrote RGB565 pixels into the 320x240 framebuffer.




```
`timescale 1ns / 1ps
```

```
// Test the top-level capture path with a short synthetic RGB565 camera line.
// The SCCB controller is bypassed so the test can focus on pixel packing,
// horizontal downsampling, framebuffer writes, and frame-ready detection.
module tb_top_capture_filters;
    reg CLK100MHZ = 1'b0;
    reg [7:0] sw = 8'h00;
    reg [7:0] cam_d = 8'h00;
    reg cam_vsync = 1'b0;
    reg cam_href = 1'b0;
    reg cam_pclk_buf = 1'b1;
    wire cam_xclk;
    wire cam_reset;
    wire cam_pwdn;
    wire cam_sioc;
    tri1 cam_siod;
    wire [3:0] vgaRed;
    wire [3:0] vgaGreen;
    wire [3:0] vgaBlue;
    wire Hsync;
    wire Vsync;
    wire [15:0] led;
    wire [11:0] framebuffer0_probe;
    wire [11:0] framebuffer1_probe;

    always #5 CLK100MHZ = ~CLK100MHZ;

    assign framebuffer0_probe = dut.framebuffer[0];
    assign framebuffer1_probe = dut.framebuffer[1];

    top_ov7670_fifo_vga dut (
        .CLK100MHZ(CLK100MHZ),
        .sw(sw),
        .cam_d(cam_d),
        .cam_vsync(cam_vsync),
        .cam_href(cam_href),
        .cam_pclk_buf(cam_pclk_buf),
        .cam_xclk(cam_xclk),
        .cam_reset(cam_reset),
        .cam_pwdn(cam_pwdn),
        .cam_sioc(cam_sioc),
        .cam_siod(cam_siod),
        .vgaRed(vgaRed),
        .vgaGreen(vgaGreen),
        .vgaBlue(vgaBlue),
        .Hsync(Hsync),
        .Vsync(Vsync),
        .led(led)
```

```
);

task send_cam_byte;
    input [7:0] value;
    begin
        cam_d = value;
        cam_href = 1'b1;
        cam_vsync = 1'b0;
        #10 cam_pclk_buf = 1'b0;
        #10 cam_pclk_buf = 1'b1;
        #10;
    end
endtask

task send_blank_pclk;
    input href_value;
    input vsync_value;
    begin
        cam_href = href_value;
        cam_vsync = vsync_value;
        #10 cam_pclk_buf = 1'b0;
        #10 cam_pclk_buf = 1'b1;
        #10;
    end
endtask

initial begin
    $dumpfile("tb_top_capture_filters.vcd");
    $dumpvars(0, tb_top_capture_filters.CLK100MHZ);
    $dumpvars(0, tb_top_capture_filters.sw);
    $dumpvars(0, tb_top_capture_filters.cam_d);
    $dumpvars(0, tb_top_capture_filters.cam_vsync);
    $dumpvars(0, tb_top_capture_filters.cam_href);
    $dumpvars(0, tb_top_capture_filters.cam_pclk_buf);
    $dumpvars(0, tb_top_capture_filters.vgaRed);
    $dumpvars(0, tb_top_capture_filters.vgaGreen);
    $dumpvars(0, tb_top_capture_filters.vgaBlue);
    $dumpvars(0, tb_top_capture_filters.led);
    $dumpvars(0, tb_top_capture_filters.dut.latched_d);
    $dumpvars(0, tb_top_capture_filters.dut.latched_href);
    $dumpvars(0, tb_top_capture_filters.dut.latched_vsync);
    $dumpvars(0, tb_top_capture_filters.dut.byte_phase);
    $dumpvars(0, tb_top_capture_filters.dut.byte_hi);
    $dumpvars(0, tb_top_capture_filters.dut.cap_x);
    $dumpvars(0, tb_top_capture_filters.dut.cap_y);
    $dumpvars(0, tb_top_capture_filters.dut.cap_dst_x);
    $dumpvars(0, tb_top_capture_filters.dut.cap_dst_y);
    $dumpvars(0, tb_top_capture_filters.dut.wr_addr);
```

```
$dumpvars(0, tb_top_capture_filters.dut.frame_ready);
$dumpvars(0, tb_top_capture_filters.dut.frame_seen);
$dumpvars(0, tb_top_capture_filters.dut.pixel_seen);
$dumpvars(0, tb_top_capture_filters.framebuffer0_probe);
$dumpvars(0, tb_top_capture_filters.framebuffer1_probe);

force dut.cam_init_done = 1'b1;

repeat (4) send_blank_pclk(1'b0, 1'b0);

// Four RGB565 source pixels. The capture logic stores source pixels
// 0 and 2 because it downsamples horizontally by 2.
send_cam_byte(8'hF8); send_cam_byte(8'h00); // red   -> framebuffer[0]
if (dut.byte_phase != 1'b0 || dut.cap_x != 10'd1 || dut.framebuffer[0] != 12'hF00) begin
    $display("FAIL: red pixel capture failed. byte_phase=%b cap_x=%0d framebuffer[0]=%h",
             dut.byte_phase, dut.cap_x, dut.framebuffer[0]);
    $finish;
end

send_cam_byte(8'h07); send_cam_byte(8'hE0); // green -> skipped
if (dut.cap_x != 10'd2 || dut.framebuffer[1] === 12'h0F0) begin
    $display("FAIL: green pixel should have been skipped. cap_x=%0d framebuffer[1]=%h",
             dut.cap_x, dut.framebuffer[1]);
    $finish;
end

send_cam_byte(8'h00); send_cam_byte(8'h1F); // blue  -> framebuffer[1]
if (dut.byte_phase != 1'b0 || dut.cap_x != 10'd3 || dut.framebuffer[1] != 12'h00F) begin
    $display("FAIL: blue pixel capture failed. byte_phase=%b cap_x=%0d framebuffer[1]=%h",
             dut.byte_phase, dut.cap_x, dut.framebuffer[1]);
    $finish;
end

send_cam_byte(8'hFF); send_cam_byte(8'hFF); // white -> skipped

send_blank_pclk(1'b0, 1'b0);
send_blank_pclk(1'b0, 1'b1); // frame boundary: marks frame ready
send_blank_pclk(1'b0, 1'b0);

if (dut.framebuffer[0] != 12'hF00) begin
    $display("FAIL: framebuffer[0] = %h, expected F00", dut.framebuffer[0]);
    $finish;
end

if (dut.framebuffer[1] != 12'h00F) begin
    $display("FAIL: framebuffer[1] = %h, expected 00F", dut.framebuffer[1]);
    $finish;
end
```

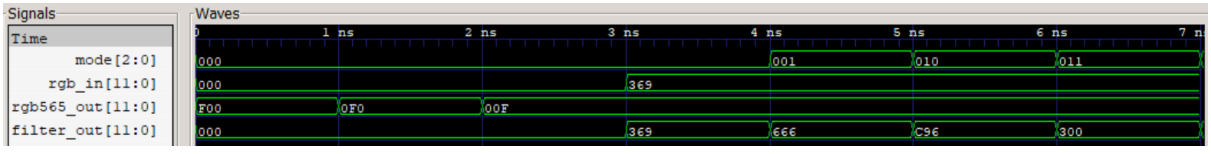
```
        if (dut.frame_ready != 1'b1) begin
            $display("FAIL: frame_ready was not set.");
            $finish;
        end

        $display("PASS: top-level capture wrote RGB565 pixels into the 320x240 framebuffer.");
        $finish;
    end
endmodule
```

---

tb_top_filter_functions : Tests color conversion and image filter helper functions.

- ➢ It checks RGB565-to-RGB444 conversion:
  - F8 00 -> F00 red
  - 07 E0 -> 0F0 green
  - 00 1F -> 00F blue
- ➢ Then it checks apply_filter() using: rgb_in = 12'h369
- ➢ Expected filter outputs:
  - mode 000 normal -> 12'h369
  - mode 001 grayscale -> 12'h666
  - mode 010 invert -> 12'hC96
  - mode 011 red only -> 12'h300
  - mode 100 green only-> 12'h060
  - mode 101 blue only -> 12'h009




```
`timescale 1ns / 1ps

// Test the required combinational image helper functions in top_ov7670_fifo_vga.
// This intentionally avoids VGA timing so the waveform stays short and the
// expected values are easy to inspect.
module tb_top_filter_functions;
    reg CLK100MHZ = 1'b0;
```

```
reg [7:0] sw = 8'h00;
reg [7:0] cam_d = 8'h00;
reg cam_vsync = 1'b0;
reg cam_href = 1'b0;
reg cam_pclk_buf = 1'b1;
wire cam_xclk;
wire cam_reset;
wire cam_pwdn;
wire cam_sioc;
tri1 cam_siod;
wire [3:0] vgaRed;
wire [3:0] vgaGreen;
wire [3:0] vgaBlue;
wire Hsync;
wire Vsync;
wire [15:0] led;

reg [11:0] rgb_in = 12'h000;
reg [2:0] mode = 3'b000;
reg [11:0] rgb565_out = 12'h000;
reg [11:0] filter_out = 12'h000;

always #5 CLK100MHZ = ~CLK100MHZ;

top_ov7670_fifo_vga dut (
    .CLK100MHZ(CLK100MHZ),
    .sw(sw),
    .cam_d(cam_d),
    .cam_vsync(cam_vsync),
    .cam_href(cam_href),
    .cam_pclk_buf(cam_pclk_buf),
    .cam_xclk(cam_xclk),
    .cam_reset(cam_reset),
    .cam_pwdn(cam_pwdn),
    .cam_sioc(cam_sioc),
    .cam_siod(cam_siod),
    .vgaRed(vgaRed),
    .vgaGreen(vgaGreen),
    .vgaBlue(vgaBlue),
    .Hsync(Hsync),
    .Vsync(Vsync),
    .led(led)
);

task check_filter;
    input [11:0] value;
    input [2:0] filter_mode;
    input [11:0] expected;
```

```
    begin
        rgb_in = value;
        mode = filter_mode;
        filter_out = dut.apply_filter(value, filter_mode);
        #1;
        if (filter_out != expected) begin
            $display("FAIL: apply_filter(%h, %b) = %h, expected %h",
                     value, filter_mode, filter_out, expected);
            $finish;
        end
    end
endtask

initial begin
    $dumpfile("tb_top_filter_functions.vcd");
    $dumpvars(0, tb_top_filter_functions.rgb_in);
    $dumpvars(0, tb_top_filter_functions.mode);
    $dumpvars(0, tb_top_filter_functions.rgb565_out);
    $dumpvars(0, tb_top_filter_functions.filter_out);

    rgb565_out = dut.rgb565_to_rgb444(8'hF8, 8'h00);
    #1;
    if (rgb565_out != 12'hF00) begin
        $display("FAIL: RGB565 red conversion = %h, expected F00", rgb565_out);
        $finish;
    end

    rgb565_out = dut.rgb565_to_rgb444(8'h07, 8'hE0);
    #1;
    if (rgb565_out != 12'h0F0) begin
        $display("FAIL: RGB565 green conversion = %h, expected 0F0", rgb565_out);
        $finish;
    end

    rgb565_out = dut.rgb565_to_rgb444(8'h00, 8'h1F);
    #1;
    if (rgb565_out != 12'h00F) begin
        $display("FAIL: RGB565 blue conversion = %h, expected 00F", rgb565_out);
        $finish;
    end

    check_filter(12'h369, 3'b000, 12'h369); // normal
    check_filter(12'h369, 3'b001, 12'h666); // grayscale
    check_filter(12'h369, 3'b010, 12'hC96); // invert
    check_filter(12'h369, 3'b011, 12'h300); // red only
    check_filter(12'h369, 3'b100, 12'h060); // green only
    check_filter(12'h369, 3'b101, 12'h009); // blue only
```

```
        $display("PASS: RGB565 conversion and image filter helpers produced expected values.");
        $finish;
    end
endmodule
```

Challenge Faced

- It was difficult to understand the OV7670 camera documentation because the camera has many registers and configuration options.
- Working with multiple clock domains, such as camera PCLK, FPGA CLK100MHZ, and VGA clk25 can be confusing.
- Tuning the camera color and brightness was a major challenge because small changes to OV7670 register values could strongly affect the final image.
- Removing red noise was difficult.
- It takes time to generate the bitstreams.

AI usage

- Generated magic mathematical filter function for RGB configuration (Color balance, Soft desaturate).
- Extracted specific hexadecimal configuration from OV7670, SCCB, Basys3 document for save time on reading.
- Debugging & Synthesis Analysis when errors occur to make more understanding of the problem.
- Scaffolded testbench structure and test case for reducing time consuming. (e.g. clock and reset loops)
- Comment on the module, variable and function.

Extra Credit: 1280x960 Bilinear Video Upscaling

For the extra-credit part, the original 320x240 camera image was upscaled to a 1280x960 VGA output using bilinear interpolation. The baseline project already captured the camera image into a 320x240 RGB444 framebuffer and displayed it on a VGA monitor by repeating each pixel in a 2x2 block for 640x480 output. For the extra credit, this was changed to a 1280x960 VGA timing mode, and the simple repeated-pixel scaling was replaced with a non-trivial bilinear scaler.

Bilinear upscaling estimates each output pixel by blending the four nearest pixels from the lower-resolution source image. For a source image pixel group:

P00 P10
P01 P11

The output pixel is calculated using its fractional horizontal and vertical position between those four pixels. First, the scaler interpolates horizontally between P00 and P10, and also between P01 and P11. Then it interpolates vertically between those two intermediate results. This creates smoother transitions than nearest-neighbor scaling, where each source pixel is simply copied into a larger block.

In this project, the source framebuffer is still 320x240, but the displayed output is 1280x960. This is a 4x scale in both directions. Therefore, each output coordinate maps back to a source coordinate like this:

```
src_x  = sx[10:2];
src_y  = sy[9:2];
frac_x = sx[1:0];
frac_y = sy[1:0];
```

The upper bits select the source pixel, and the lower two bits select the fractional position inside the 4x4 expanded region.

Implementation

The framebuffer was not enlarged to 1280x960. That would require too much memory. Instead, we keep the original 320x240 RGB444 framebuffer and perform the upscaling while reading pixels for VGA output.

A new module (bilinear_upscaler_4x.v) was added. This module reads source pixels from the framebuffer and outputs a bilinear-interpolated RGB444 pixel for each 1280x960 VGA coordinate.

Because bilinear interpolation needs four neighboring pixels, the scaler uses row buffers. It stores two source rows at a time:

row0 = current source row
row1 = next source row

For each output pixel, it reads:

P00 = row0[src_x]
P10 = row0[src_x + 1]
P01 = row1[src_x]
P11 = row1[src_x + 1]

Then it performs interpolation separately for the red, green, and blue channels. Since the scale factor is exactly 4x, the fractional weights are only 0/4, 1/4, 2/4, 3/4
This means the interpolation can be implemented using only adders and bit shifts.

Clock and VGA Timing Changes

The original project generated 640x480 VGA timing using a 25 MHz pixel clock. For the extra credit, the VGA timing was changed to 1280x960 60 Hz, which requires a 108 MHz pixel clock.

A new module (vga_clock_108.v) was added. It uses a Xilinx MMCM to generate a 108 MHz pixel clock from the Basys 3's 100 MHz board clock.

The original 25 MHz clock is still kept for the OV7670 camera XCLK, because the camera configuration was already working with that clock.

Files Changed

top_ov7670_fifo_vga.v

Main changes:

- Added the 108 MHz VGA clock.

```
46  45          reg [1:0] div4 = 2'b00;
47  46          always @(posedge CLK100MHZ) div4 <= div4 + 2'd1;
48  47
49  48          wire clk25 = div4[1];
    49  +       wire clk108;
    50  +       wire clk108_locked;
    51  +
    52  +       vga_clock_108 clk108_u (
    53  +           .clk100 (CLK100MHZ),
    54  +           .clk108 (clk108),
    55  +           .locked (clk108_locked)
    56  +       );
```

```
    -       wire [9:0] sx, sy;
75  +       wire [10:0] sx;
76  +       wire [9:0]  sy;
77          wire        active_video;
78
79          vga_640x480 vga_u (
    -           .clk25          (clk25),
80  +           .clk25          (clk108),
81              .sx             (sx),
82              .sy             (sy),
83              .hsync          (Hsync),
```

- Connected the new bilinear scaler.

```
168       -     wire [8:0] src_x = sx[9:1];
169       -     wire [7:0] src_y = sy[8:1];
170       -     wire [16:0] rd_addr =
171       -         ({9'd0, src_y} << 8) + ({9'd0, src_y} << 6) + {8'd0, src_x};
      176 +     wire [16:0] scaler_fb_addr;
      177 +     reg  [11:0] scaler_fb_data = 12'h000;
      178 +     wire [11:0] scaled_rgb;
      179 +     wire        scaled_valid;
      180 +
      181 +     always @(posedge clk108) begin
      182 +         scaler_fb_data <= framebuffer[scaler_fb_addr];
      183 +     end
      184 +
      185 +     bilinear_upscaler_4x scaler_u (
      186 +         .clk          (clk108),
      187 +         .frame_ready  (frame_ready),
      188 +         .sx           (sx),
      189 +         .sy           (sy),
      190 +         .active_video (active_video),
      191 +         .fb_addr      (scaler_fb_addr),
      192 +         .fb_data      (scaler_fb_data),
      193 +         .rgb          (scaled_rgb),
      194 +         .valid        (scaled_valid)
      195 +     );
```

- Kept the 320x240 framebuffer.
- Changed the VGA output path to use the interpolated RGB value instead of simple 2x pixel repetition.

```
346       -         if (active_video && frame_ready) begin
347       -             camera_rgb <= framebuffer[rd_addr];
      370 +         if (active_video && scaled_valid && clk108_locked) begin
      371 +             camera_rgb <= scaled_rgb;
```

```
359       -     assign vgaRed   = active_video ? (sw[0] ? test_rgb[11:8] : (sw[1] ? live_rgb[11:8] : filtered_rgb[11:8])) : 4'h0;
360       -     assign vgaGreen = active_video ? (sw[0] ? test_rgb[7:4]  : (sw[1] ? live_rgb[7:4]  : filtered_rgb[7:4]))  : 4'h0;
361       -     assign vgaBlue  = active_video ? (sw[0] ? test_rgb[3:0]  : (sw[1] ? live_rgb[3:0]  : filtered_rgb[3:0]))  : 4'h0;
      383 +     assign vgaRed   = (active_video && clk108_locked) ? (sw[0] ? test_rgb[11:8] : (sw[1] ? live_rgb[11:8] : filtered_rgb[11:8])) : 4'h0;
      384 +     assign vgaGreen = (active_video && clk108_locked) ? (sw[0] ? test_rgb[7:4]  : (sw[1] ? live_rgb[7:4]  : filtered_rgb[7:4]))  : 4'h0;
      385 +     assign vgaBlue  = (active_video && clk108_locked) ? (sw[0] ? test_rgb[3:0]  : (sw[1] ? live_rgb[3:0]  : filtered_rgb[3:0]))  : 4'h0;
```

vga_640x480.v

Main changes:

- Changed the size of sx register

```
4       9     module vga_640x480 (
5       10        input  wire       clk25,
6         -       output reg [9:0]  sx = 0,
        11  +     output reg [10:0] sx = 0,
7       12        output reg [9:0]  sy = 0,
8       13        output wire       hsync,
9       14        output wire       vsync,
10      15        output wire       active_video,
11      16        output wire       frame_start
12      17    );
```

- Updated timing from 640x480 to 1280x960.
- Changed horizontal total to 1800 pixels.
- Changed vertical total to 1000 lines.

```
17  22      always @(posedge clk25) begin
18      -       if (sx == 799) begin
        23  +       if (sx == 1799) begin
19  24              sx <= 0;
20      -           if (sy == 524) sy <= 0;
        25  +       if (sy == 999) sy <= 0;
21  26              else           sy <= sy + 1;
22  27          end else begin
23  28              sx <= sx + 1;
24  29          end
25  30      end
```

- Changed sync pulses to match 1280x960 @ 60 Hz timing.

```
27      -   assign hsync = ~((sx >= 656) && (sx < 752));
28      -   assign vsync = ~((sy >= 490) && (sy < 492));
29      -   assign active_video = (sx < 640) && (sy < 480);
        32  + assign hsync = (sx >= 1376) && (sx < 1488);
        33  + assign vsync = (sy >= 961) && (sy < 964);
        34  + assign active_video = (sx < 1280) && (sy < 960);
30  35      assign frame_start  = (sx == 0) && (sy == 0);
```

New Files Added

1. bilinear_upscaler_4x.v
2. vga_clock_108.v

bilinear_upscaler_4x.v: Implements the 4x bilinear interpolation hardware.

```
`timescale 1ns / 1ps

// ============================================================================
// โมดูลขยายภาพแบบ Bilinear 4 เท่า (320x240 -> 1280x960)
//
// คอนเซปต์หลัก: แทนที่จะอ่าน RAM หลัก 4 ครั้งต่อ 1 พิกเซล (ซึ่ง Clock วิ่งไม่ทันแน่ๆ)
// โมดูลนี้จะแอบไป "ดูดข้อมูลล่วงหน้า (Prefetch)" มาเก็บไว้ใน "ถังพักข้อมูล (Row Buffers)" ทีละ 2 บรรทัด
// โดยมีถังพัก 2 ชุดสลับกันทำงาน (Bank A และ Bank B) เพื่อให้สตรีมภาพออกจอได้ลื่นไหลไม่มีสะดุด
// ============================================================================
module bilinear_upscaler_4x (
    input  wire        clk,          // นาฬิกา 108 MHz สำหรับจอ HD
    input  wire        frame_ready,  // สัญญาณบอกว่ามีภาพเต็มเฟรมพร้อมให้ดึงแล้ว
    input  wire [10:0] sx,           // พิกัด X บนหน้าจอ VGA (0-1279)
    input  wire [9:0]  sy,           // พิกัด Y บนหน้าจอ VGA (0-959)
    input  wire        active_video, // 1 = อยู่ในช่วงวาดภาพลงจอ
    output reg  [16:0] fb_addr = 17'd0, // ตำแหน่ง Address ที่ส่งไปขอข้อมูลจาก Framebuffer หลัก
    input  wire [11:0] fb_data,      // ข้อมูลสีพิกเซลที่ Framebuffer หลักตอบกลับมา
    output wire [11:0] rgb,          // สีที่ผ่านการผสม (Bilinear) เสร็จแล้ว ส่งออกไปรอขึ้นจอ
    output wire        valid         // 1 = ข้อมูลสี rgb พร้อมใช้งานแล้ว
);
    localparam FRAME_WIDTH  = 320;
    localparam FRAME_HEIGHT = 240;

    // ========================================================================
    // ส่วนที่ 1: หน่วยความจำชั่วคราว (Row Buffers - Double Buffering)
```

```
// ใช้ Distributed RAM ใน FPGA สร้างที่เก็บข้อมูลทีละ 1 บรรทัด (320 พิกเซล)
// แบ่งเป็น Bank A และ Bank B (แต่ละ Bank เก็บแถวบน row0 และแถวล่าง row1 เพื่อใช้ผสมสีแนวตั้ง)
// ============================================================================
(* ram_style = "distributed" *) reg [11:0] row0_a [0:FRAME_WIDTH-1];
(* ram_style = "distributed" *) reg [11:0] row1_a [0:FRAME_WIDTH-1];
(* ram_style = "distributed" *) reg [11:0] row0_b [0:FRAME_WIDTH-1];
(* ram_style = "distributed" *) reg [11:0] row1_b [0:FRAME_WIDTH-1];

// ตัวแปรควบคุมการสลับ Bank (ตัวไหนกำลังส่งออกจอ, ตัวไหนกำลังไปดูดข้อมูลใหม่)
reg current_bank = 1'b0;
reg current_valid = 1'b0;
reg [7:0] current_y = 8'd0;

reg bank_valid_a = 1'b0;
reg bank_valid_b = 1'b0;
reg [7:0] bank_y_a = 8'd0;
reg [7:0] bank_y_b = 8'd0;

// ============================================================================
// ส่วนที่ 2: State Machine สำหรับดึงข้อมูลล่วงหน้า (Prefetch Logic)
// ============================================================================
reg prefetch_busy = 1'b0;
reg prefetch_bank = 1'b0;
reg [7:0] prefetch_y = 8'd0;
reg [8:0] prefetch_x = 9'd0;
reg       prefetch_row = 1'b0;
reg       rd_pending = 1'b0;
reg       pending_bank = 1'b0;
reg       pending_row = 1'b0;
reg [8:0] pending_x = 9'd0;

// คำนวณว่าตอนนี้หน้าจอกำลังวาดอยู่ที่ Y ตันฉบับบรรทัดไหน (เอา sy มาหาร 4 โดยการตัด 2 บิตล่างทิ้ง)
wire [7:0] display_y = sy[9:2];

// คำนวณหาบรรทัดถัดไปที่ต้องไปดูดข้อมูลมารอไว้ (ถ้ายังไม่สุดขอบล่าง ก็ดึงบรรทัดถัดไปมารอเลย)
wire [7:0] desired_prefetch_y =
    (!active_video && (sy >= 10'd960)) ? 8'd0 :
    (display_y == 8'd239) ? 8'd239 : (display_y + 8'd1);

// เช็คว่า Bank ไหนมีข้อมูลของบรรทัดที่หน้าจอกำลังวาดอยู่
wire bank_a_matches_display = bank_valid_a && (bank_y_a == display_y);
wire bank_b_matches_display = bank_valid_b && (bank_y_b == display_y);

// เลือก Bank ที่จะเอามาใช้งานผสมสีส่งออกจอ
wire selected_bank = bank_a_matches_display ? 1'b0 :
                     bank_b_matches_display ? 1'b1 : current_bank;

// กำหนด Bank ที่ว่างอยู่ ให้วิ่งไป Prefetch ข้อมูลมารอ
wire prefetch_target_bank = bank_a_matches_display ? 1'b1 :
```

```
                        bank_b_matches_display ? 1'b0 : ~current_bank;

    wire any_bank_has_desired =
        (bank_valid_a && (bank_y_a == desired_prefetch_y)) ||
        (bank_valid_b && (bank_y_b == desired_prefetch_y));

    // ฟังก์ชันช่วยคำนวณ Address 1D จากพิกัด 2D (สูตร Y*320 + X แบบใช้ Shift ประหยัดตัวคูณ)
    function [16:0] make_addr;
        input [7:0] y;
        input [8:0] x;
        begin
            make_addr = ({9'd0, y} << 8) + ({9'd0, y} << 6) + {8'd0, x}; // Y*256 + Y*64 + X
        end
    endfunction

    // บล็อกการทำงานหลัก: จัดการดึงข้อมูลจาก Framebuffer หลักมาใส่ Row Buffer ตัวที่ว่างอยู่
    always @(posedge clk) begin
        if (rd_pending) begin
            // เอาข้อมูลที่ตอบกลับมาจาก RAM ยัดลง Bank ที่เลือกไว้
            if (!pending_bank) begin
                if (!pending_row) row0_a[pending_x] <= fb_data;
                else              row1_a[pending_x] <= fb_data;
            end else begin
                if (!pending_row) row0_b[pending_x] <= fb_data;
                else              row1_b[pending_x] <= fb_data;
            end

            rd_pending <= 1'b0;
            // ดึงครบทั้งบรรทัด (319) แล้วหรือยัง?
            if (pending_row && (pending_x == FRAME_WIDTH-1)) begin
                prefetch_busy <= 1'b0; // ดึงครบ 2 แถว (row0, row1) แล้ว แจ้งว่า Bank นี้พร้อมใช้งาน
                if (!pending_bank) begin
                    bank_valid_a <= 1'b1;
                    bank_y_a <= prefetch_y;
                end else begin
                    bank_valid_b <= 1'b1;
                    bank_y_b <= prefetch_y;
                end
            end else if (pending_x == FRAME_WIDTH-1) begin
                prefetch_row <= 1'b1; // ดึงแถวแรกเสร็จแล้ว สลับไปดึงแถวที่สอง (row1) ต่อ
                prefetch_x <= 9'd0;
            end else begin
                prefetch_x <= pending_x + 9'd1; // ขยับไปพิกเซลต่อไปแนวนอน
            end
        end else if (prefetch_busy) begin
            // ส่ง Address ไปขอข้อมูลจาก RAM หลัก
            fb_addr <= make_addr(prefetch_row ? ((prefetch_y == 8'd239) ? 8'd239 : prefetch_y + 8'd1) :
prefetch_y, prefetch_x);
            pending_bank <= prefetch_bank;
```

```
            pending_row <= prefetch_row;
            pending_x <= prefetch_x;
            rd_pending <= 1'b1; // รอ RAM ตอบกลับใน Clock ถัดไป

        end else if (frame_ready && !any_bank_has_desired) begin
            // เริ่มต้นกระบวนการ Prefetch สำหรับบรรทัดใหม่
            prefetch_busy <= 1'b1;
            prefetch_bank <= prefetch_target_bank;
            prefetch_y <= desired_prefetch_y;
            prefetch_x <= 9'd0;
            prefetch_row <= 1'b0;
            if (!prefetch_target_bank) bank_valid_a <= 1'b0;
            else                       bank_valid_b <= 1'b0;
        end

        // อัปเดตสถานะ Bank ปัจจุบันจังหวะที่หน้าจอเริ่มปัดหัวอ่านกลับมาต้นบรรทัด
        if (active_video && (sx == 11'd0) && (sy[1:0] == 2'b00)) begin
            if (!current_bank && bank_valid_a && (bank_y_a == display_y)) begin
                current_valid <= 1'b1;
                current_y <= display_y;
            end else if (current_bank && bank_valid_b && (bank_y_b == display_y)) begin
                current_valid <= 1'b1;
                current_y <= display_y;
            end else if (!current_bank && bank_valid_b && (bank_y_b == display_y)) begin
                current_bank <= 1'b1; // สลับ Bank!
                current_valid <= 1'b1;
                current_y <= display_y;
            end else if (current_bank && bank_valid_a && (bank_y_a == display_y)) begin
                current_bank <= 1'b0; // สลับ Bank!
                current_valid <= 1'b1;
                current_y <= display_y;
            end else begin
                current_valid <= 1'b0;
            end
        end
    end

    // ==========================================================================
    // ส่วนที่ 3: คณิตศาสตร์ Bilinear Interpolation (ผสมสี 4 จุด)
    // ==========================================================================
    wire [8:0] raw_src_x = sx[10:2]; // เอาพิกัด X จอใหญ่ มาหาร 4 เพื่อหาตำแหน่งพิกเซลต้นฉบับ

    // ป้องกัน Noise ตรงขอบจอด้านซ้าย (แก้ปัญหาขอบภาพสั่น)
    wire [8:0] src_x = (raw_src_x == 9'd0) ? 9'd1 : raw_src_x;

    // หาตำแหน่งพิกเซลถัดไปทางขวา (X+1) โดยต้องไม่เกินขอบขวาสุด (319)
    wire [8:0] src_x1 = (src_x == 9'd319) ? 9'd319 : (src_x + 9'd1);

    // เอาเศษจากการหาร 4 (คือ 2 บิตล่าง) มาเป็นค่าน้ำหนัก (Fraction: 0, 1, 2, 3) เพื่อดูว่าจุดผสมอยู่ใกล้พิกเซลไหน
```

```
มากกว่ากัน
    wire [1:0] frac_x = sx[1:0];
    wire [1:0] frac_y = sy[1:0];

    // ดึงสีของจุด 4 มุม มาเตรียมไว้
    // p00 (ซ้ายบน), p10 (ขวาบน)
    // p01 (ซ้ายล่าง), p11 (ขวาล่าง)
    wire [11:0] p00 = !selected_bank ? row0_a[src_x]  : row0_b[src_x];
    wire [11:0] p10 = !selected_bank ? row0_a[src_x1] : row0_b[src_x1];
    wire [11:0] p01 = !selected_bank ? row1_a[src_x]  : row1_b[src_x];
    wire [11:0] p11 = !selected_bank ? row1_a[src_x1] : row1_b[src_x1];

    // ------------------------------------------------------------------
    // ฟังก์ชันวิเศษ: ผสมสีแนวนอน 2 จุด (Horizontal Mix) แบบไม่ต้องใช้ Multiplier!
    // การ Shift ซ้าย 1 ที (<< 1) คือการคูณ 2 ในทางดิจิทัล
    // ------------------------------------------------------------------
    function [5:0] hmix4;
        input [3:0] a;
        input [3:0] b;
        input [1:0] frac;
        begin
            case (frac)
                2'd0: hmix4 = {a, 2'b00};                                  // ตรงเป๊ะที่พิกเซลซ้าย: เอา A มา 100% (A * 4)
                2'd1: hmix4 = ({2'b00, a} << 1) + {2'b00, a} + {2'b00, b};       // ขยับมา 1/4: เอา A มา 3 ส่วน, B มา 1
ส่วน (2A + A + B = 3A+1B)
                2'd2: hmix4 = ({2'b00, a} << 1) + ({2'b00, b} << 1);             // อยู่ตรงกลางเป๊ะ 2/4: เอา A มา 2 ส่วน, B
มา 2 ส่วน (2A + 2B)
                default: hmix4 = {2'b00, a} + ({2'b00, b} << 1) + {2'b00, b};    // ขยับมา 3/4 (ใกล้จุดขวา): เอา A มา 1
ส่วน, B มา 3 ส่วน (1A + 3B)
            endcase
        end
    endfunction

    // ------------------------------------------------------------------
    // ฟังก์ชันผสมสีแนวตั้ง 2 จุด (Vertical Mix)
    // เอาผลลัพธ์แนวนอนบน-ล่างมาผสมกันอีกที เพื่อให้เสร็จสมบูรณ์
    // ------------------------------------------------------------------
    function [3:0] vmix4;
        input [5:0] top_scaled;
        input [5:0] bottom_scaled;
        input [1:0] frac;
        reg [7:0] scaled16;
        begin
            case (frac)
                2'd0: scaled16 = {top_scaled, 2'b00};
                2'd1: scaled16 = ({2'b00, top_scaled} << 1) + {2'b00, top_scaled} + {2'b00, bottom_scaled};
                2'd2: scaled16 = ({2'b00, top_scaled} << 1) + ({2'b00, bottom_scaled} << 1);
                default: scaled16 = {2'b00, top_scaled} + ({2'b00, bottom_scaled} << 1) + {2'b00,
bottom_scaled};
```

```
            endcase
            vmix4 = scaled16[7:4]; // หั่นบิตออก 4 บิต (เทียบเท่ากับการหาร 16 เพื่อคืนค่ากลับมาเป็นเลข 4 บิตเหมือนเดิม)
        end
    endfunction

    // ทำการผสมสีทีละชาแนล (แยก แดง, เขียว, น้ำเงิน ทำอิสระต่อกัน)
    // 1. ผสมแนวนอนแถวบน
    wire [5:0] r_top = hmix4(p00[11:8], p10[11:8], frac_x);
    // 2. ผสมแนวนอนแถวล่าง
    wire [5:0] r_bot = hmix4(p01[11:8], p11[11:8], frac_x);
    // (ทำแบบเดียวกันกับ Green และ Blue)
    wire [5:0] g_top = hmix4(p00[7:4], p10[7:4], frac_x);
    wire [5:0] g_bot = hmix4(p01[7:4], p11[7:4], frac_x);
    wire [5:0] b_top = hmix4(p00[3:0], p10[3:0], frac_x);
    wire [5:0] b_bot = hmix4(p01[3:0], p11[3:0], frac_x);

    // 3. เอาแถวบนกับแถวล่างมาผสมกันในแนวตั้ง ประกอบร่างกลับเป็น RGB 12 บิต!
    assign rgb = {
        vmix4(r_top, r_bot, frac_y),
        vmix4(g_top, g_bot, frac_y),
        vmix4(b_top, b_bot, frac_y)
    };

    // ตรวจสอบความถูกต้องว่าข้อมูลพร้อมส่งออกพอร์ต VGA
    assign valid = frame_ready && active_video &&
                   ((bank_a_matches_display && !selected_bank) ||
                    (bank_b_matches_display && selected_bank));
endmodule
```

vga_clock_108.v: Generates the 108 MHz VGA pixel clock using an MMCM.

```
`timescale 1ns / 1ps

// ================================================================================
// โมดูลแปลงความถี่นาฬิกา (Clock Generator)
// จาก 100 MHz (ของบอร์ด Basys 3) -> เป็น 108 MHz (สำหรับจอ VGA 1280x960)
// ================================================================================
module vga_clock_108 (
    input  wire clk100, // นาฬิกาต้นทางจากคริสตัลบนบอร์ด (100 MHz)
    output wire clk108, // นาฬิกาปลายทางที่ปั่นเสร็จแล้ว (108 MHz)
    output wire locked  // สัญญาณบอกสถานะ (1 = ความถี่นิ่งและพร้อมใช้งานแล้ว)
);

// ================================================================================
// ส่วนที่ 1: โหมดจำลองการทำงาน (Simulation Bypass)
// ================================================================================
// ถ้าเรากำลังรันจำลองการทำงาน (Simulate) ด้วยโปรแกรม Icarus Verilog
// โปรแกรมมันจะไม่รู้จักฮาร์ดแวร์ MMCM ของ Xilinx โค้ดส่วนนี้จึงสั่งให้เชื่อมสายตรงเลย
```

```
    // (อนุโลมให้ใช้ 100 MHz แทน 108 MHz ไปก่อนตอนทำ Simulation เพื่อให้คอมไพล์ผ่าน)
    `ifdef __ICARUS__
        assign clk108 = clk100;
        assign locked = 1'b1;
    `else

    // ============================================================================
    // ส่วนที่ 2: โหมดใช้งานจริงบนบอร์ด (Hardware Synthesis)
    // ============================================================================
        wire clkfb_unbuf;     // สาย Feedback ก่อนเข้า Buffer
        wire clkfb;           // สาย Feedback หลังเข้า Buffer (เอาไว้ให้วงจรเช็คว่าความถี่นิ่งหรือยัง)
        wire clk108_unbuf;    // สาย 108 MHz ดิบๆ ที่เพิ่งปั่นเสร็จ

        // เรียกใช้ IP ของ Xilinx (MMCME2_BASE) ซึ่งเป็นวงจรคูณ/หารความถี่ (PLL)
        // ---------------------------------------------------------------------
        // **สมการคณิตศาสตร์การปั่น Clock ของ Xilinx:**
        // F_out = F_in * (M / (D * O))
        //
        // F_in (CLKIN1) = 100 MHz (คาบ 10.000 ns)
        // D (DIVCLK_DIVIDE) = หาร 5  -> ได้ความถี่ฐาน = 20 MHz
        // M (CLKFBOUT_MULT) = คูณ 54 -> ได้ความถี่แกนกลาง (VCO) = 1080 MHz
        // O (CLKOUT0_DIVIDE)= หาร 10 -> ได้ความถี่ส่งออก (F_out) = 108 MHz เป๊ะ!
        // ---------------------------------------------------------------------
        MMCME2_BASE #(
            .BANDWIDTH("OPTIMIZED"),
            .CLKFBOUT_MULT_F(54.000),   // ตัวคูณ (Multiplier = M)
            .CLKFBOUT_PHASE(0.000),
            .CLKIN1_PERIOD(10.000),     // คาบเวลาขาเข้า 10ns = 100 MHz
            .CLKOUT0_DIVIDE_F(10.000),  // ตัวหารขาออก (Output Divider = O)
            .CLKOUT0_DUTY_CYCLE(0.500), // อัตราส่วน High/Low แบบ 50:50
            .CLKOUT0_PHASE(0.000),
            .DIVCLK_DIVIDE(5),          // ตัวหารขาเข้า (Input Divider = D)
            .REF_JITTER1(0.010),        // ค่าเผื่อความแกว่งของสัญญาณ
            .STARTUP_WAIT("FALSE")
        ) mmcm_u (
            .CLKIN1(clk100),        // ป้อน 100 MHz เข้าไป
            .CLKFBIN(clkfb),        // รับสาย Feedback กลับเข้ามาเช็คความแม่นยำ
            .CLKFBOUT(clkfb_unbuf), // ปล่อยสาย Feedback ออกไป
            .CLKOUT0(clk108_unbuf), // ปล่อยผลลัพธ์ 108 MHz ออกมาทางช่อง 0
            .CLKOUT1(), .CLKOUT2(), .CLKOUT3(), .CLKOUT4(), .CLKOUT5(), .CLKOUT6(), // ช่องอื่นไม่ได้ใช้
            .LOCKED(locked),        // ขา Output บอกว่าตอนนี้ความถี่นิ่ง 108 MHz แล้วนะ
            .PWRDWN(1'b0),          // ไม่เปิดโหมดประหยัดพลังงาน
            .RST(1'b0)              // ไม่ต้องรีเซ็ต (ทำงานทันทีที่เปิดเครื่อง)
        );

        // ============================================================================
        // ส่วนที่ 3: จัดการสายสัญญาณนาฬิกา (Global Clock Buffers)
        // ============================================================================
        // สัญญาณนาฬิกาเป็นสัญญาณที่ซีเรียส ต้องเดินทางไปถึงทุกชิ้นส่วนพร้อมกัน
```

```
    // เราจึงต้องเอาสัญญาณดิบไปผ่าน BUFG (Global Clock Buffer)
    // เพื่อดันสัญญาณให้วิ่งเข้าไปใน "ท่อพิเศษสำหรับ Clock" ของชิป FPGA

    // จับ 108 MHz ดิบๆ ยัดเข้า BUFG
    BUFG clk108_bufg_u (
        .I(clk108_unbuf),
        .O(clk108) // กลายเป็น clk108 ที่แข็งแรงและพร้อมใช้งาน!
    );

    // จับสัญญาณ Feedback ยัดเข้า BUFG
    // (เพื่อให้ MMCM คำนวณความหน่วงของวงจรได้อย่างแม่นยำ)
    BUFG clkfb_bufg_u (
        .I(clkfb_unbuf),
        .O(clkfb)
    );
`endif
endmodule
```